

MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC OF
KAZAKHSTAN

INTERNATIONAL INFORMATION TECHNOLOGY UNIVERSITY

FACULTY OF COMPUTER TECHNOLOGY AND CYBERSECURITY

Arapov Ibragim
Sanbayeva Symbat
Saulebayev Ernur

Detection of Fake Accounts in a Corporate Network (Active Directory)

DIPLOMA PROJECT

Educational Program: 6B06303 – Network Security

Almaty 2026

MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE REPUBLIC OF
KAZAKHSTAN

INTERNATIONAL INFORMATION TECHNOLOGY UNIVERSITY

FACULTY OF COMPUTER TECHNOLOGIES AND CYBERSECURITY

DEPARTMENT OF CYBERSECURITY

Approved

Head of Department,
cand. of tech. sc., assoc. professor
D.M. Yeskendirova

«_____» _____ 2026

DIPLOMA PROJECT

Detection of Fake Accounts in a Corporate Network (Active Directory)

Education program 6B06303-Network security

Done by students:

Arapov I.K. _____ «_____» _____ 2026
(signature)

Sanbayeva S.B. _____ «_____» _____ 2026
(signature)

Saulebayev E.G. _____ «_____» _____ 2026
(signature)

Research advisor:

Beginbayeva E.E. _____ «_____» _____ 2026
(signature)

Almaty 2026

International Information Technology University
Faculty of Computer Technology and Cybersecurity
Department of Cybersecurity

Educational program 6B06303 – Network Security

Diploma Project Assignment

Students: **Arapov Ibragim, Sanbayeva Symbat, Saulebayev Ernur**

Diploma Project Topic: « **Detection of Fake Accounts in a Corporate Network (Active Directory)** »

Approved by IITU order № _____ dated « _____ » _____ 2026

Diploma work (project) submission date « _____ » _____ 2026

Diploma Project initial data

Develop and test a Python-based software tool for detecting fake accounts in a corporate Active Directory network by analyzing Windows Security logs exported in JSON/JSONL format. The system should normalize event data, correlate security events around account creation and activation, identify suspicious indicators, calculate risk levels, and generate investigation reports in HTML, CSV, and JSON formats.

Details of computations and explanations

(list of issues due to be addressed)

1. Analysis of the role of Active Directory in corporate network security.
2. Study of Windows Security Logs, Event Viewer, EVTX format, and key Event IDs related to account activity.
3. Analysis of methods used by attackers to create fake accounts in Active Directory environments.
4. Study of common attacker tools and techniques, including Kerberos-related attacks, privilege escalation, Mimikatz, BloodHound, and PowerShell-based activity.
5. Identification of diagnostic indicators of fake accounts, including account creation outside business hours, suspicious usernames, rapid privilege escalation, and Kerberos ticket requests after account creation.
6. Definition of functional requirements for a fake account detection tool.
7. Design of the program architecture for importing logs, normalizing data, building account-based cases, and calculating risk scores.
8. Development of a Python-based log parser for JSON and JSONL Windows Security log files.
9. Implementation of event correlation logic based on account creation, activation, password changes, group membership changes, logon events, and Kerberos

activity.

10. Development of a weighted risk assessment model for classifying suspicious accounts by severity level.
11. Generation of investigation reports in HTML, CSV, and JSON formats for SOC analysts and Active Directory administrators.
12. Testing of the developed prototype on prepared log datasets and evaluation of its ability to detect suspicious fake account scenarios.
13. Preparation of project documentation, explanatory note, appendices, and defense presentation.

CD containing the digital version of Diploma Project and attachments

1. The source code and explanatory note;
2. Project presentation;
3. Electronic version of the diploma project;
4. Test log datasets and generated reports.

Consultations on diploma work (project) (with related project chapters named)

Consultant	Name	Signature, date	
		Assignment given	Assignment received
Compliance monitor	<i>Sh.N.Makilenov,</i> <i>senior-lecturer</i>	«_____»_____2026	«_____»_____2026
		_____	_____

Date «_____»_____2026

Research advisor

(signature)

Assignment received by

(signature)

Arapov I.K., Sanbayeva S.B., Saulebayev E.G.

Title: Detection of Fake Accounts in a Corporate Network (Active Directory)

№	Assignment	Submission date
1.	Creation of the graduation paper writing schedule	October 20
2.	Collection, study, processing, analyzing and generalizing data	November – December
3.	Writing the introduction and the first chapter	January
4.	Pre-defence	February 13
5.	Completing the 2nd major thesis and the first biometrics testing	March
6.	Revision of the 1st and 2nd theses with due consideration of the comments of the supervisor	March 31-4
7.	Solving problems with the anti-spoofing system and biometrics	April 6-15
8.	Writing 3 chapters and making adjustments with the supervisor	April 16-30
9.	Preparing to submit a thesis project for anti-plagiarism	May 1-15
10.	Submission of the diploma paper for the plagiarism check-up	May 16
11.	Checking the standard control	May 28
12.	Diploma work (project) defense	June 2

Student: Arapov I.K.

(signature)

Student: Sanbayeva S.B.

(signature)

Student: Saulebayev E.G.

(signature)

Research advisor: Begimbayeva E.E.

(signature)

Date « ____ » _____ 20 ____

АҢДАТПА

Дипломдық жобаның мақсаты-Active Directory көмегімен корпоративті желідегі жалған шоттарды тану бағдарламасын құру. Жалпы Siem жүйелері мен журналдарды қолмен талдау көбінесе шағын және орта ұйымдар үшін тиімсіз немесе тым қымбат болғандықтан, арнайы ашық бастапқы шешімдер іс жүзінде жоқ. Бұл жоба Windows қауіпсіздік журналдарын талдайтын және жалған есептік жазбалардың белгілерін анықтайтын бағдарламалық жасақтаманы құрудағы алғашқы жұмыс. Сонымен бірге біз evtx журналдарының архитектурасын, оқиғалардың негізгі идентификаторларын, kerberoasting, Golden Ticket және т.б. сияқты заманауи шабуыл құрылымдарын, сондай-ақ шабуылдаушылардың жабдықтарын зерттедік. Тереңдетілген зерттеу негізінде диагностиканың негізгі белгілері анықталды: жұмыс уақытынан тыс уақытта тіркелгі жасау, артықшылықты топтарға жылдам қосу, күдікті атаулар және тіркелгі жасалғаннан кейін бірден Kerberos сұраулары. Python-дағы бағдарлама JSON/JSONL форматында экспортталған журналдарды оқиды, деректерді қалыпқа келтіреді, әр есептік жазбаның айналасында корпус модельдерін құру, талдау үшін бағалау моделі және HTML, CSV және JSON форматындағы визуалды есеп арқылы жағдай жасайды. Журналдардың сынақ жиынтықтарында прототиптің күдікті сценарийлерді анықтау қабілетін растау үшін эксперименттік сынақтар жүргізілді. Осылайша, әзірленген құрал ұйымдарда да, оқу және зерттеу мақсаттарында да қолданылады.

Дипломдық жобада 69 бет, 22 кесте, 15 сурет, 50 сілтеме және 1 қосымша берілген.

Кілттік сөздер: ЖАЛҒАН АККАУНТТАРДЫ АНЫҚТАУ, ACTIVE DIRECTORY, WINDOWS SECURITY LOGS, EVTX, EVENT ID, KERBEROS, SIEM, PYTHON, JSON/JSONL, КОРПОРАТИВТІК ЖЕЛІ ҚАУІПСІЗДІГІ.

АННОТАЦИЯ

Целью дипломного проекта является создание программы для распознавания поддельных аккаунтов в корпоративной сети с использованием Active Directory. Поскольку общие системы SIEM и ручной анализ логов часто неэффективны или слишком дороги для малых и средних организаций, а специализированных решений с открытым исходным кодом практически нет. Этот проект является первой работой по созданию программного обеспечения, которое анализирует журналы безопасности Windows и выявляет признаки фейковых аккаунтов. В то же время мы изучили архитектуру логов EVTX, основные идентификаторы событий, современные структуры атак, такие как Kerberoasting, Golden Ticket и т.д., а также оборудование атакующих. На основе углубленного исследования были выявлены основные признаки диагностики: создание аккаунта в нерабочее время, быстрое добавление в привилегированные группы, подозрительные имена и запросы Kerberos сразу после создания аккаунта. Программа на Python читает экспортированные логи в формате JSON/JSONL, нормализует данные, создает кейс вокруг каждого аккаунта через построение моделей кейсов, модель оценки для анализа и визуальный отчет в формате HTML, CSV и JSON. На тестовых наборах логов были проведены экспериментальные тесты для подтверждения способности прототипа выявлять подозрительные сценарии. Таким образом, разработанный инструмент будет применим как в организациях, так и для учебных и исследовательских целей.

В дипломном проекте представлены 69 страниц, 22 таблицы, 15 рисунков, 50 источников и 1 приложение.

Ключевые слова: ОБНАРУЖЕНИЕ ФЕЙКОВЫХ АККАУНТОВ, ACTIVE DIRECTORY, WINDOWS SECURITY LOGS, EVTX, EVENT ID, KERBEROS, SIEM, PYTHON, JSON/JSONL, БЕЗОПАСНОСТЬ КОРПОРАТИВНОЙ СЕТИ.

ABSTRACT

The goal of the project is to create a program for detecting fake accounts in a corporate network using Active Directory. Since general SIEM systems and manual protocol analysis are often inefficient or too expensive for small and medium-sized organizations, there are practically no specialized open source solutions. This project is the first work on creating a software that analyzes the Windows security protocols and identifies signs of fake accounts. At the same time, we examined the architecture of the EVTX protocols, basic event identifiers, modern attack structures such as kerberoasting, golden ticket, etc., as well as the attackers' equipment. Based on in-depth research, the main diagnostic features were identified: creating an account outside of business hours, quickly adding to privileged groups, suspicious names and Kerberos queries immediately after creating an account. The Python program reads the exported logs in JSON/JSONL format, normalizes the data, creates a case around each account by creating case models, creating an evaluation model for analysis, and creating a visual report in HTML, CSV, and JSON formats. The test sets of the protocols were subjected to experimental tests to confirm the prototype's ability to detect suspicious scenarios. Therefore, the developed tool will be suitable for organizations, as well as for educational and research purposes.

The diploma project contains 69 pages, 22 tables, 15 figures, 50 references, and 1 appendix.

Keywords: FAKE ACCOUNT DETECTION, ACTIVE DIRECTORY, WINDOWS SECURITY LOGS, EVTX, EVENT ID, KERBEROS, SIEM, PYTHON, JSON/JSONL, CORPORATE NETWORK SECURITY.

CONTENTS

	LIST OF TERMS AND ABBREVIATIONS	8
	INTRODUCTION	9
1	THEORETICAL PART	10
1.1	Windows OS Overview from a Cybersecurity Perspective	10
1.2	Windows Logging System Architecture and EVTX Format	10
1.3	Methods for Creating Fake Accounts and Tools Used by Attackers	14
1.4	Goal of Project	22
2	DESIGN AND METHODOLOGY	24
2.1	Purpose and Role of Fake Accounts	24
2.2	Program Code Explanation	28
3	PRACTICAL PART: DETAILED PROJECT DEVELOPMENT	37
3.1	Practical Visualization of How the Program Works Part 1	37
3.2	Practical Visualization of How the Program Works Part 2	43
	CONCLUSION	61
	REFERENCES	62
	APPENDIX – Code listing	66

LIST OF TERMS AND ABBREVIATIONS

Active Directory — a Microsoft directory service used to manage users, computers, groups, policies, and other objects in a corporate network.

AD DS — Active Directory Domain Services, a directory service that stores information about network objects and provides access to this information for authorized users and administrators.

Domain — a logical group of users, computers, and resources managed under the same Active Directory security and administrative policies.

Domain Controller — a server that stores Active Directory data and handles authentication, authorization, and directory service requests within a domain.

User Account — an identity object used to authenticate a person or service and provide access to network resources.

Fake Account — an unauthorized or suspicious account created or activated in a corporate environment for illegitimate access or persistence.

Privileged Account — an account that has elevated permissions and can perform administrative or security-sensitive actions in the system.

Security Group — an Active Directory group used to assign permissions and control access to resources.

Windows Security Log — a Windows log category that records security-related events such as logons, account management, privilege use, and policy changes.

Event Viewer — a Windows administrative tool used to view and analyze system, application, and security logs.

EVTX — the Windows Event Log file format used to store event records generated by the operating system and applications.

Event ID — a numerical identifier assigned to a Windows event that describes the type of activity recorded in the log.

Audit Policy — a Windows security configuration that defines which types of activities should be recorded in security logs.

SIEM — Security Information and Event Management, a system used to collect, correlate, analyze, and monitor security events from different sources.

SOC — Security Operations Center, a team or department responsible for monitoring, detecting, analyzing, and responding to cybersecurity incidents.

Kerberos — a network authentication protocol used in Windows domain environments to verify the identity of users and hosts.

TGT — Ticket Granting Ticket, a Kerberos ticket issued after successful authentication and used to request access to services.

TGS — Ticket Granting Service, a Kerberos service that issues service tickets for access to specific network resources.

Privilege Escalation — a technique or process by which an attacker gains higher-level permissions than originally assigned.

Risk Score — a numerical value used by the developed system to estimate the suspiciousness and severity of an account-related case.

INTRODUCTION

The timeliness of this topic is due to the fact that the number of cyber attacks aimed at companies with Active Directory, is increasing, and we no longer have the reliable approach to the detection of fake accounts. This topic interests us when we have learned about new and modern cyberbullying. After studying the reports, it became clear that many attacks are related to compromising credentials. If an attacker has access, he has first tried to attach him, thereby creating a new account or activating old forgotten. However, this is one of the more subtle, but also the most penetrating strategies. If an attacker accesses the account long enough, because the account resembles an ordinary worker, no one will follow him. Modern organizations are improving the security of their domain infrastructure, but as our analysis shows, many means of protection, such as antiviruses, standard audit policies and SIEM systems, are not always able to effectively deal with the identification of fake accounts. This vulnerability has been fixed in our program. Unlike the counterparts, it connects events with important cases, provides a simple risk assessment for each account and provides a visible justification for why such an account is suspicious. That is why this specific, albeit small, important threat makes it worthwhile to develop a specialized tool for threats.

Assess the current state of the problem we are solving. And at the moment the problem of detection of fake accounts in Active Directory is an open question. The vast majority of organizations still use standard Windows tools, and some of them implement expensive SIEM platforms, such as Splunk or Qradar. However, each of these approaches has its drawbacks. The first approach involves constant manual analysis and depends on the skills of the expert. The second approach is associated with high costs, which most medium-sized companies cannot afford, and besides, it is poorly suited for identifying fake accounts and usually produces false positive results. Therefore, the current state of the problem is characterized by the absence of special open solutions, as well as a high threshold of entry for organizations with a limited budget.

The practical significance of our work is that the software developed by us can be used for the activities of security operations centers (SOC) or for the daily work of Active Directory administrators. The main difference of our program in practice is that it tracks the time sequence and creates an overall picture around the account. This is useful for regular audits as a stand-alone tool or as an additional tool for existing monitoring systems. We leave the source code open and the architecture is modular, so that any expert can adapt it to the needs of his company. Therefore, one of the unique advantages of our program is that it is suitable for both large companies and small businesses that cannot afford expensive SIEM solutions.

The aim of the study is an Active Directory-based corporate network of the Windows operating system.

The subject of the study is methods and algorithms for detecting fake accounts, based on the analysis of security protocols and their correlation with time contexts, IP addresses, security groups and behavioral anomalies.

1 THEORETICAL PART

1.1 The relevance of the security problem of the Windows operating system

The architecture of the Windows operating system has undergone numerous changes over the past few years. Each new version has its own unique features. Its security architecture is improved, stable, and more comprehensive, making the system the best it can be [1]. However, no system can be completely secure. Our team studied Windows from various angles, examining system issues that could impact its functionality. Windows itself, the Mitre attack organization, and other IT companies and organizations, particularly those focused on system security, periodically provide security updates to these structures. For example, we studied the new OWASP 2025 and concluded that almost half of these vulnerabilities are closely related to Windows architecture [50]. Windows is now in widespread use, and for this reason, all attacks target the Windows operating system. Our team studied the available number of Windows users, including active and passive ones, and their percentage was over 90%. This figure includes all devices running the Windows operating system, not just desktop computers and laptops. Personal computers account for approximately 72 percent of all Windows operating systems, especially the new flagship Windows 11. This large number suggests that perhaps every third Windows operating system is potentially compromised or hacked and serves as a botnet. Based on our research, we concluded that vulnerabilities and compromised PCs are the direct result of user incompetence or negligence. However, it's important to note that this isn't entirely the user's problem; external and internal factors can play a role. My team, working as a pentester, discovered that vulnerabilities in web applications, such as web platforms and websites, can also contribute to the compromise of the Windows operating system. Examples include SQL injection, command injection, API injection, and other types of vulnerabilities [50]. Owasp publishes vulnerability data annually. However, this data only shows the most commonly used vulnerabilities among hackers. Other vulnerabilities shouldn't be ignored, as they can also be present within the web application architecture. This demonstrates that external resources are closely intertwined with the Windows ecosystem [2]. Internal factors: the Windows operating system's internal development itself can be vulnerable or compromised at the architectural level due to improper assembly. Update patches are also a factor that can influence the compromise of the Windows operating system. Our team identified all of these vulnerabilities using materials provided by Microsoft itself and with the help of other private experts [1, 7].

1.2 Windows Logging System Architecture and EVTX Format

The primary goal of this project was to study logging in the Windows operating system. Each operating system has its own logging format. In Windows, this is the EVTX format. Initially, before EVTX, the evt format was used. However, over time

and the evolution of the Windows operating system, the logging format also changed to EVTX, which stands for Windows XML Event Log [7].

Windows logging, specifically the Event log, collects data from every running process in the Windows operating system. Event Viewer is a software component of the Windows operating system architecture. Event Viewer handles the administration and management of logs within the Windows operating system and on Windows-based servers that include Active Directory [1]. Windows operating system logs include all processes that have started or have been started, such as PC startup, login and password entry, application launch, network logon, updates, deletions, and creations. Even simple events like opening File Explorer are also logged. Logs are collected using Event Log. It internally collects all logs, and Event Viewer then divides them into different categories based on severity and log content. There are system logs, security logs, application logs, and other logs, and they are divided into categories. Each log records exactly what falls within its category. The screenshot shows how the Event Viewer interface works. On the left side, you'll find Windows logs, which include Application, Security, Setup, System, and Forwarded Events. There are also application logs, service logs, and Custom logs. Each of these operates within its own service area, allowing for smoother administration than a mixed log system [46]. If logs were mixed, the system would have difficulty distributing them, and this would also lead to analysis failures for the network administrator or cybersecurity analyst.

Figure 1.1 shows the structure of the Event Viewer interface. Windows 10 and Windows 11 share the same interface and core functionality, which, according to the study, results in smoother use compared to older versions of the Windows operating system.

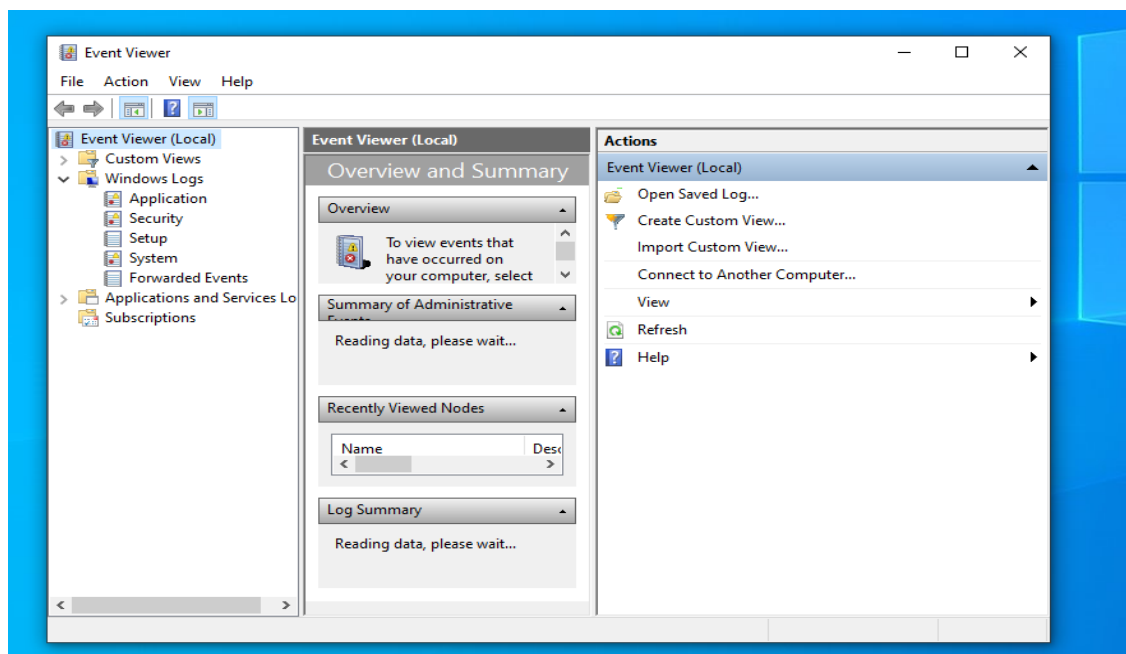


Figure 1.1 - Event Viewer

In Figures 1.2 and 1.3, you can clearly see the differences between the categories

The screenshot shows the Windows Event Viewer application. The left-hand pane displays the tree view of event logs, with 'Security' selected under 'Windows Logs'. The main pane shows the 'Security' log, which contains 192 events. The first 10 events are visible, all of which are 'Audit' events. The selected event is 5379, 'Microsoft Windows security auditing'. The details pane for this event is expanded, showing the following information:

- General**
 - Credential Manager credentials were read.**
 - Subject:** Security ID: DANIEL\peter
 - Log Name:** Security
 - Source:** Microsoft Windows security
 - Event ID:** 5379
 - Task Category:** User Account Management
 - Level:** Information
 - Keywords:** Audit Success
 - User:** N/A
 - Computer:** Daniel

The right-hand pane shows the 'Actions' pane, which includes options such as 'Open Saved Log...', 'Create Custom View...', 'Import Custom View...', 'Clear Log...', 'Filter Current Log...', 'Properties', 'Find...', 'Save All Events As...', 'Attach a Task to This Log...', 'View', 'Refresh', 'Help', and 'Event 5379: Microsoft Windows security auditing...'.

Event Viewer

File Action View Help

Event Viewer (Local)

Custom Views

Administrative Events

Windows Logs

Application

Security

Setup

System

Forwarded Events

Applications and Services Logs

Subscriptions

Application

Number of events: 3 950

Level	Date and Time	Source	Event ID	Task Category
Information	24.03.2026 16:49:51	Security-SPP	16384	None
Information	24.03.2026 16:49:14	Security-SPP	16394	None
Information	24.03.2026 16:35:36	edgeupdate	0	None
Information	24.03.2026 16:17:25	Security-SPP	16384	None
Information	24.03.2026 16:16:55	Security-SPP	16394	None
Information	24.03.2026 16:11:08	OneDriveUpdaterService	0	None
Information	24.03.2026 16:11:00	OneDriveUpdaterService	0	None
Information	24.03.2026 16:01:35	Security-SPP	16384	None
Information	24.03.2026 16:01:05	Security-SPP	16394	None
Error	24.03.2026 15:43:11	.NET Runtime	1022	None
Information	24.03.2026 15:31:31	Security-SPP	16384	None
Information	24.03.2026 15:31:01	Security-SPP	16394	None
Information	24.03.2026 15:11:18	Security-SPP	16384	None
Information	24.03.2026 15:10:48	Security-SPP	16394	None
Information	24.03.2026 15:00:16	Security-SPP	16384	None
Information	24.03.2026 14:59:46	Security-SPP	16394	None
Information	24.03.2026 14:50:32	Security-SPP	16384	None
Information	24.03.2026 14:50:02	Security-SPP	16394	None
Information	24.03.2026 14:34:21	Security-SPP	16384	None

Event 16384, Security-SPP

General

Details

Successfully scheduled Software Protection service for re-start at 2026-03-25T05:44:51Z. Reason: RulesEngine.

Log Name:

Source:

Event ID:

Level:

User:

OpCode:

Application

Security-SPP

16384

Information

N/A

Info

Logged:

Task Category:

Keywords:

Computer:

24.03.2026 16:49:51

None

Classic

Daniel

Actions

Application

Open Saved Log...

Create Custom View...

Import Custom View...

Clear Log...

Filter Current Log...

Properties

Find...

Save All Events As...

Attach a Task To This Log...

View

Refresh

Help

Event 16384, Security-SPP

Event Properties

Attach Task To This Event...

Copy

Save Selected Events...

Refresh

Help

13

Since the topic of this project is fake accounts, the primary focus should be on the Security section of Windows logging. This section is particularly important because it is directly related to Windows Defender [2]. Security is a special section of Windows logs that records security-related data, logs, and processes. These can include authentication, deletion, or creation of security-critical processes. While not critical by default, everything depends on the timing of these processes and the algorithm that created the chain of events. The basis of each process is the chain of events before and after. And as mentioned earlier, this section is also regulated by audit policy to separate noise from critical logs. Another important category within the Security section is Event ID. This is a combination of numbers used to identify the type of event within the Windows ecosystem [9, 14]. However, it is not unique to everyone. Event ID depends on the source of the event. A disk, applications, or something else within the ecosystem can have radically different descriptions. A key advantage of Event ID is that it does not provide a textual description, only a numerical identifier. Further values can be read in the official Microsoft documentation.

Since the research and practical work of this project is based on the implementation of fake account detection, it's worth demonstrating how an event works within Windows. Figures 1.3 and 1.4 show an event with a source and event ID. The source is VMnetDHCP, and Event ID 1 indicates a WHEA-Logger error. Based on document analysis, this event explains that there was no data from some component within the system, which is why Event ID 1 was generated. And here's where one problem arises. Without full context, it's impossible to understand exactly what happened. This topic has already been described previously: the importance of the chain, not a single event. A single event can only provide a view from one context, but the full picture of what's happening. SOC analysts and cybersecurity analysts are investigating precisely this interpretation [47].



Figure 1.4 - Application logs (zoomed)

1.3 Methods for creating fake accounts and tools used by attackers

Investigating fake accounts in Active Directory environments is crucial, as this specific incident is extremely challenging for attackers and also dangerous for defenders and the corporate Active Directory environment itself. Understanding what a fake account is is crucial. First, we need to distinguish between legitimate and

illegitimate fake accounts. A legitimate fake account is when network administrators create a fake account to catch hackers. Hackers often fall for such fake accounts, also known as honeypots [38]. However, this project will focus specifically on investigating fake accounts as an illegitimate component. This will begin with a review of articles on the purposes of attackers creating fake accounts in Active Directory environments. These purposes include finance, data, and data manipulation.

The Figure 1.5 and 1.6 show a possible example of creating a fake account. These images show the account creation log. They are taken from the Event Viewer, which was discussed in previous threads.

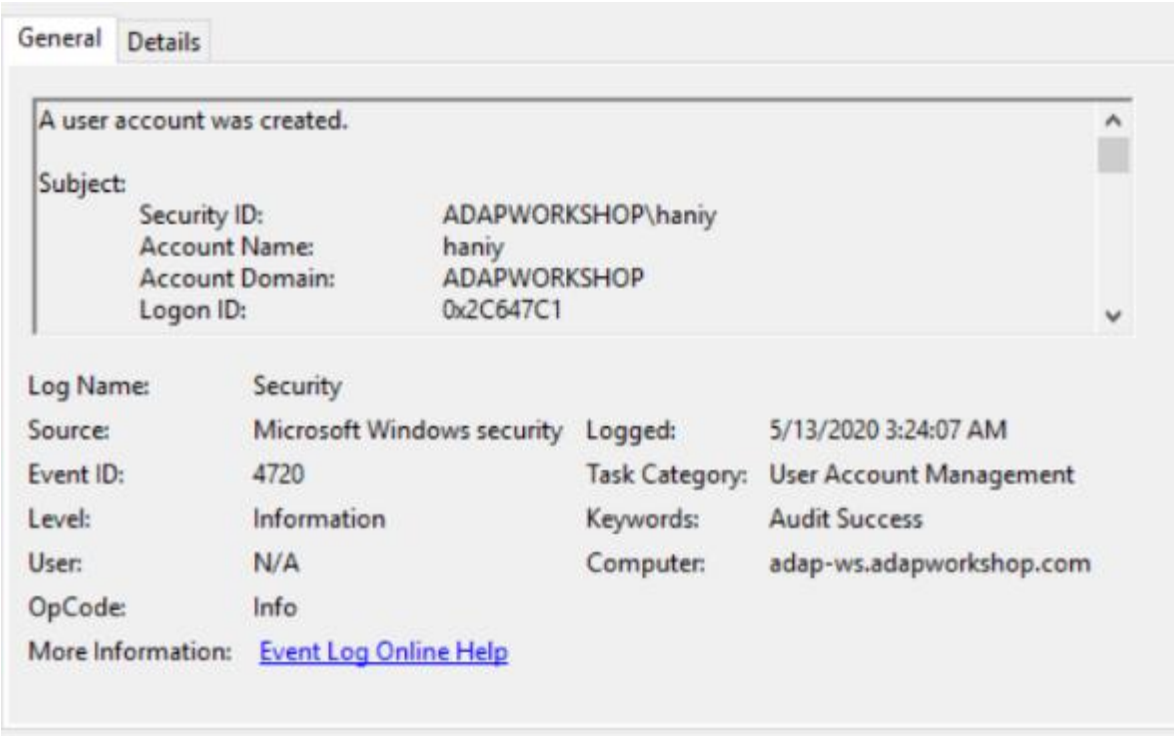


Figure 1.5 - Log screenshot from the Even Viewer

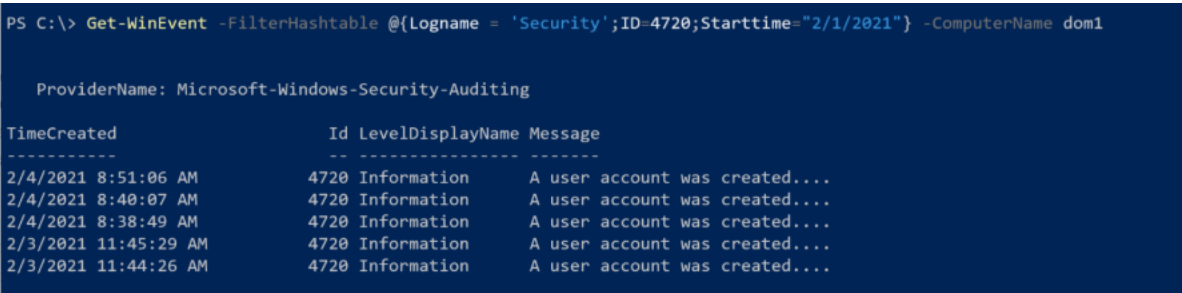


Figure 1.6 - Logs of created new account from Powershel

These logs are collected using event logs. The network administrator sets a security and audit policy for a specific time and pattern. This pattern should record abnormal network authentications and anomalous account creations at anomalous times. However, security and audit policies are not absolute protection. For this reason,

additional security measures such as antivirus software and SIEM tools are always necessary [33, 46]. Penetration testing is also worth mentioning.

Based on research, pentesters are a vital part of the ecosystem, providing attack overviews, attack results, and systematizing attacker methods. They follow the Kill Chain algorithm. Pentesters use various types of tools and also use their own scripts or those of other authors. The difference between tools and scripts is that tools provide a broader and more general range of attack implementations, while scripts provide highly targeted attack methods based on a specific system architecture.

Based on research, hackers' tools and methods for creating fake accounts vary widely. One such method is the Privilege Escalation attack [41]. This attack method allows an attacker to gain access from the initial level to a higher level within the Active Directory system. This method allows an attacker to change audit policies, modify permissions, lock functionality, add or delete files, and change network access levels, making the corporate system vulnerable to various types of attacks. In this case, the attacker will be able to modify the Active Directory structure itself. These changes can disrupt Active Directory operation to such an extent that an illegitimate individual can secure a place within the system for future login access.

An attacker's initial steps are to gain initial access to a corporate system. Initial access is obtained from various sources [50]. Some platforms, such as GitHub, may have leaked company data, including credentials. This data may be outdated, but some companies, particularly administrators, may forget to remove a user from the Active Directory network. Another method of exploiting this is initial access. This occurs when an attacker gains access within the network using either leaked data or social engineering. Although social engineering may not sound critical, it is dangerous enough to pose challenges for cybersecurity analysts. On a corporate network, social engineering is one of the most dangerous attacks. With social engineering, the primary vulnerability is the individual, the company employee. For this reason, security managers conduct training to keep company employees informed about attacks and how they operate. If any strange email is detected, cybersecurity analysts are immediately notified. Communication between departments is also important in cybersecurity. An attacker's primary goal when accessing a corporate network is to find an unused account. As noted, unused accounts are often not monitored, so the attacker's primary goal is an unused account. Such an account can later become a fake account. Or, if such an account doesn't exist in the Active Directory network, the attacker targets more vulnerable attack vectors. These are guests or users of the corporate Active Directory network. Low-level users, namely regular entry-level employees who use Active Directory, are the most vulnerable. According to the research and experience of the research group conducting this thesis, using an active account for illegal purposes is highly undesirable in good hacking practice. Since these accounts are monitored 24/7 and the audit policy configured by administrators targets unusual account patterns, this is a natural setting for corporate networks. Therefore, creating a fake account or using an unused account is one of the safest ways to bypass AD security. The problem is that on a corporate network, a low-level account doesn't have the right to create accounts, and not every network administrator has the right to create an account. Administrator

privileges on a corporate network have their own hierarchy and their own process domain that they are authorized to use. According to research by Microsoft, IBM, and other IT giants, these rules for delegating processes and services between administrators are the best practice for maintaining the security of an intranet. If a single administrator account had full rights to everything, compromising that account would completely destroy the entire security system of the corporate network. Finding an administrator account to create a fake account with administrator privileges is an extremely difficult task for both a pentester and an attacker. The purpose of creating a fake administrator account is to give it the most privileges to manipulate the entire corporate Active Directory system. Increasing an account's access level within a corporate network from a low to a higher level is called privilege escalation. One method of privilege escalation is Kerberoasting (Figure 1.7) [44, 45]. It is one of the most difficult for an attacker to implement and difficult for SOC team analysts to find, as the attacker uses TGT tickets from the AD system itself on the main server to gain access to processes and services within the corporate network, thereby escalating their access. The goal of Kerberoasting is to find tickets for a potential victim account on the network. After dumping the tickets from the data, the attacker can then further escalate their privileges within the network. Kerberos also uses a method called Golden Ticket. This method grants unlimited access to an account controlled by the attacker. Golden Ticket allows the attacker to create unlimited access to tickets [43]. Why was the Kerberos attack specifically considered in this section? Kerberos is a new security system within a corporate Active Directory network. Its primary purpose isn't simply storing hashes in a data center, but rather issuing tickets for various processes and services. The idea is that for authentication, network authorization, and access to a shared directory in the data center, each user is issued a unique ticket, also encrypted with asymmetric encryption. The primary ticket is issued during network authentication and authorization. This ticket changes with each login; more precisely, the internal bits of this ticket change, and access to a process or service within AD also requires a ticket.

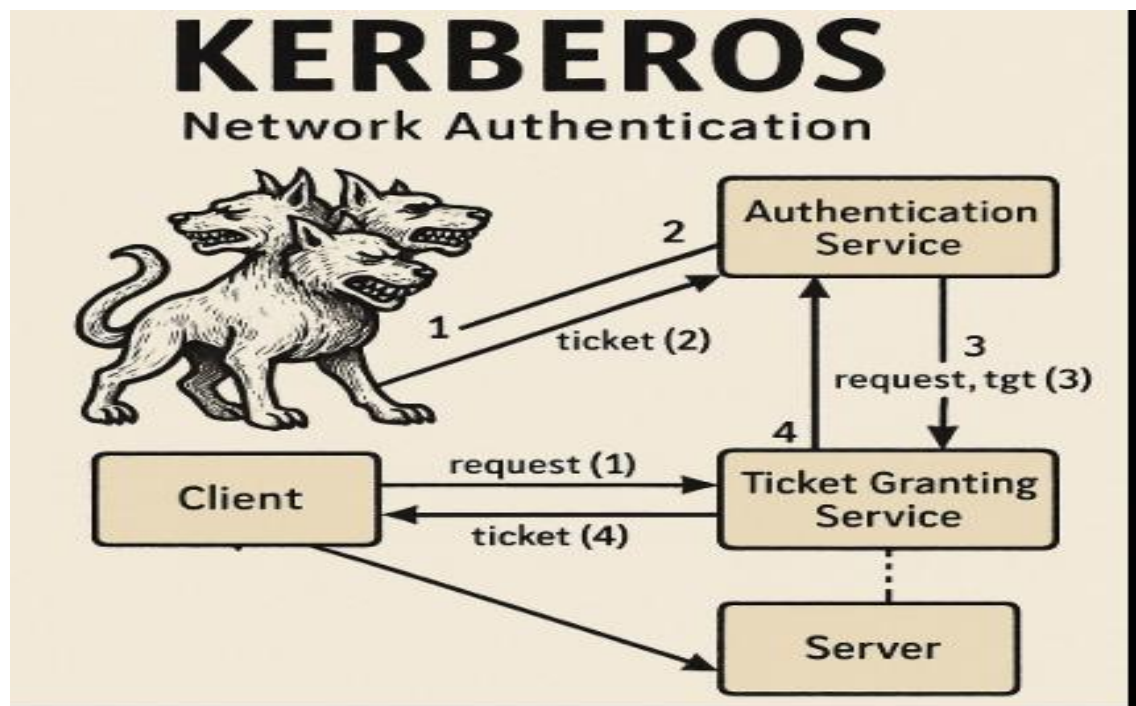


Figure 1.7 - How kerberos works in Active Directory

The basic idea behind hacking is for the attacker to steal data, manipulate data within the corporate network, and perform other tasks. The key for a pentester is to understand the attacker's logic in order to subsequently help network analysts improve the network. However, it's not just about pentesters or attackers; a number of widely used hacking tools are also involved. GitHub and some private organizations are developing their own security testing systems, but these tools are also created by attackers, often for a fee and often on the darknet. Among the publicly available tools, the GitHub library and private organizations are available. Almost everything is open source, which allows for future improvements. One of the most popular hacking tools is Mimikatz [42]. It is closely related to the topic of fake accounts, as this software specifically helps attackers steal data from corporate networks and further escalate privileges. As mentioned, privilege escalation is the method used to create fake accounts. Mimikatz employs a wide variety of hacking methods, including stealing hashes, passwords, and PINs from various accounts (Figure 1.8). A memory block—an executable file called lsass.exe—is used for the theft. This is because this file stores important data such as passwords, hashes, tokens, and other sensitive information that would be critical if it fell into the attacker's hands. For this reason, pentesters often operate with a very narrow operational scope to avoid interfering with or disrupting the system during a pentest attack. AD is extremely sensitive to critical files.

```
##### mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
## ^ ## "A La Vie, A L'Amour" - (oe,oe)
## / W ## /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## W / ## > https://blog.gentilkiwi.com/mimikatz
'## v ## Vincent LE TOUT ( vincent.letoux@gmail.com )
'##### > https://pingcastle.com / https://mysmartlogon.com ***//

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 310276 (00000000:0004bc04)
Session           : Interactive from 1
User Name          : clientUser
Domain            : AHNLAB
Logon Server       : DC01
Logon Time         : 2023-12-23 오후 1:17
SID               : S-1-5-21-2260624-1-1-495-3107
```

Figure 1.8 - Mimikatz tools that allows privilege escalation tool

Mimikatz itself was created within Microsoft to test systems for vulnerabilities, but it has evolved into a tool that is most often used for both pentesting and illegitimate attacks.

The next tool is Bloodhound. As mentioned earlier, searching for vulnerable accounts, accounts with administrator privileges that have varying levels of access, and various process and service variations, and identifying the accounts that could be useful to an attacker when creating a fake account is extremely difficult and time-consuming, as the Active Directory architecture is extremely complex, encompassing not only users but also other objects. Without a dedicated tool, it would be difficult to create a proper topology, and Bloodhound is a useful tool in this case.

As shown in Figure 1.9, Bloodhound displays a dependency graph of the internal Active Directory architecture. Bloodhound's primary function is to analyze a corporate Active Directory network and generate a graph as shown in the screenshot. It helps identify weaknesses in Active Directory and potential vectors for gaining higher levels of interaction with the ecosystem, specifically, privilege escalation. This tool is used by both penetration testers and hackers. Bloodhound's importance for this thesis is that it assists attackers in potentially creating fake accounts, but it can also assist in analyzing a corporate Active Directory network for potential vulnerabilities

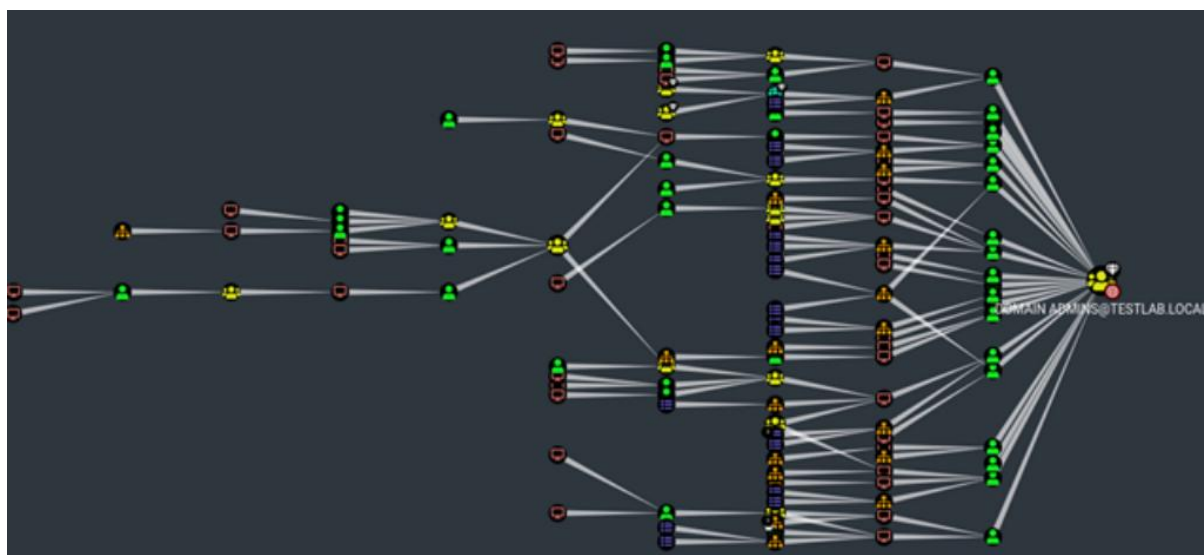


Figure 1.9 - Bloodhound tool

Another critical tool is PowerSploit (Figure 1.10). PowerSploit is a tool similar to its Metasploit counterpart that specifically targets Windows-based software. It offers the same functionality as Metasploit. Its features include reconnaissance, data theft, data manipulation, antivirus bypass, data dumping, and, most importantly, privilege escalation.

```
PS C:\Windows\system32> Get-WmiObject -class win32_OperatingSystem

SystemDirectory : C:\Windows\system32
Organization     :
BuildNumber      : 9200
RegisteredUser   : Windows User
SerialNumber     : 00183-80000-11491-AA269
Version          : 6.2.9200

PS C:\Windows\system32> (New-Object System.Net.Webclient).DownloadString('https://raw.githubusercontent.com/daveha/PowerShell-Scripts/master/in-memory-downloader.ps1')|iex

The following modules were imported into this PowerShell session;
#####
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/CodeExecution/Invoke--Shellcode.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/CodeExecution/Invoke-DllInjection.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/Exfiltration/Invoke-Mimikatz.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/Exfiltration/Invoke-NinjaCopy.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/Exfiltration/Get-GPPPassword.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/Exfiltration/Get-Keystrokes.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/Exfiltration/Get-TimedScreenshot.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/CodeExecution/Invoke-ReflectivePEInjection.ps1
https://raw.githubusercontent.com/mattifestation/PowerSploit/master/PowerUp/PowerUp.ps1
https://raw.githubusercontent.com/Veil-Framework/PowerTools/master/PowerView/powerview.ps1
#####
For detailed help on each use Get-Help ModuleName ie "Get-Help Invoke-AllChecks -Full", as sample shown below.

NAME
Invoke-AllChecks

SYNOPSIS
Runs all current Windows privesc checks.

SYNTAX
Invoke-AllChecks [<CommonParameters>]

DESCRIPTION
This function runs all functions that check for various
Windows privilege escalation opportunities.
```

Figure 1.10 - The PowerSploit interface

According to a study of current cybersecurity articles, the list of tools being used is growing, and the functionality of existing tools is also expanding. These tools are closely related to the goal of this project, as they can also help improve the software with additional features based on penetration testing data. Focusing solely on system security, while relying solely on human resources when creating security protocols, is highly unacceptable. Security must be assessed from both sides—the attacker's and the defender's.

Now, moving on to Blue Team's research-based protective methods, the system of protective measures is quite hierarchical compared to Red Team's. Protective methods require specific knowledge in a narrow area of research. These include SOC analysts, who are also divided by process level within the cybersecurity field, digital forensics experts, and network administrators. The initial link is SOC analysts, whose tools include a SIEM system. Blue Team is more complex than it seems at first glance. Cybersecurity analysts have to work with a huge data stream, much of which is potentially false positives. But even false positives must also be analyzed, as it is impossible to be absolutely certain whether an incident is positive or false. Configuring a SIEM system based on patterns and rules so that the tool collects data based on this data is essential to separate noise from real data [35]. Without proper configuration, the system can collect too much data, making it difficult for analysts to conduct an accurate analysis. Furthermore, the system itself is not designed to analyze such a large amount of data due to memory and processing speed limitations. The organization's business context must also be considered. Business logic. Although it's not typically conceptualized as an attack, it often ranks among the top vulnerabilities according to the OWASP TOP 10.

The primary task of such analysts is to read the system and logs as a whole, rather than focusing on a single event. This topic was mentioned earlier regarding the chain of events. Since a single connection within the system at an unusual time does not necessarily mean the system has been hacked, incident triage is necessary in this case. This involves searching for the chain of events before and after a specific security incident. An event leads to a security incident. Further analysis of this security incident can provide a clear understanding of what occurred. Even a single security incident can have over a thousand event logs, which must be read within the system to understand the incident. Events can come from different IP addresses and different accounts, but they can be linked by attack logic. For this reason, the primary goal is to look at the event as a whole, not just focusing on a single unusual event. Based on my experience as a SOC analyst, incident triage is an extremely labor-intensive task that requires not only looking within the system but also communicating with those the security system protects—namely, other employees. If the system is a critical incident, in our case, the creation of a fake account, then, as mentioned, it's necessary to analyze the time, event type, severity, and how it occurred, and then block the compromised account. In this case, analysts look not only at the SIEM tool but also at event logs, event IDs, and PowerShell is also a good tool. It's extremely difficult for analysts to detect fake accounts in the system, as they can hide behind legitimate AD attributes. In the pentest section, the reason for creating a fake account instead of using an already compromised

one was mentioned: monitoring monitors legitimately created accounts and the audit policy is tailored to them, but a fake account created by the attacker can prevent the system from responding to this account using the same security audit policy, allowing the attacker to log in at any time without being detected. An extremely skilled hacker can conceal their presence through various methods, and one of the most important is to avoid aggressive actions within the system to avoid being detected by the security system. In this case, SIEM tools, as mentioned above, are the best aids for analysts.

SIEM systems came into production because initially there were only two types: SIM, which collected logs from various devices, and SEM, which handled continuous monitoring. However, since this proved inconvenient, the idea of creating a SIEM system, which stands for Security Information and Management, came up. The general parameters of a SIEM function are collecting information from various devices within the system [36, 37]. These devices include domain controllers, routers, printers, servers, and other devices that are part of the network. As mentioned, every system generates logs continuously, and a SIEM system can assist in this monitoring.

In cybersecurity, there are five types of SIEM. The first is a system installed within the network, an on-premises SIEM. It can operate locally without a network connection, eliminating dependency on the external network. The entire load is handled by its own collectors. The second type is a cloud-driven SIEM, which doesn't require the entire system to be installed on servers; instead, it's installed by agents collecting telemetry, making it easy to set up. There is also a hybrid type that combines the best of both worlds. There are also types of SIEM that include additional tools such as SOAR, XDR, EDR, and so on, depending on requirements.

SIEMs have their strengths and weaknesses. One is the sheer scale of these systems. SIEM tools can collect telemetry from multiple systems and organize it into rules. They can also implement rules that help analyze anomalies using public and private playbooks and rules.

Despite these advantages, this system still has its drawbacks, one of which is related to the topic of this project: Fake Account Detection. A SIEM tool is good at scalability, but it has limitations in detecting fake accounts. A SIEM system can collect data patterns such as group policy changes, passwords, and unusual login times, but it cannot provide results from these events. For this purpose, companies like SPLUNK and QRadar are now incorporating AI into their systems [33, 34]. But even this isn't very useful, as it is also trained for scale, and then all the work falls to the analyst. Creating a fake account is a highly targeted attack. The system may suspect an anomaly, but it won't be able to provide results.

1.4 Goal of Project

The aim of the study is to create a software that can detect fake accounts in an Active Directory—based corporate network by analyzing Windows security protocols based on rules and using a risk assessment model.

To achieve this goal, you need to perform the following tasks:

- To conduct a theoretical analysis of the role of Active Directory for the security of the corporate division, study the Windows protocol architecture and evaluate the main tactics of attackers in creating fake accounts.
- Therefore, consider modern methods of detecting unusual activity in a domain and discuss its advantages and disadvantages.
- Also identify diagnostic features of a fake account, such as creating it outside business hours, adding it to privileged groups, suspicious names, etc.
- To create an algorithm that can automatically detect fake accounts based on the correlation of event chains and a risk assessment model with weighting factors.
- In addition, they are developing a Python program to read logs, normalize data, create cases, calculate risks and generate visual reports.
- Also, check if the program is working in the test log records to confirm that it can detect suspicious scenarios.

2 DESIGN AND METHODOLOGY

2.1 Purpose and Role of fake accounts.

This program was developed to help cybersecurity analysts detect fake accounts. In today's world, an account is the key to information within a system. Fake accounts are extremely difficult to detect if a hacker knows how to exploit the system's weaknesses. Every system has its strengths and weaknesses, and no system is completely secure. Therefore, every system is inherently vulnerable. Numerous monitoring systems exist for protection. These monitoring tools help correlate logs, as discussed in previous chapters. However, the problem is that tools like Splunk, Qradar, and others operate extensively [33, 34]. You can create rules of conduct and policies, but even then, the system will still be vulnerable. In this case, a program for detecting fake accounts can help. Detecting fake accounts stems from the real root of the problem. Investigating fake accounts in an Active Directory domain infrastructure is very difficult, even for advanced specialists. Detecting whether a fake account exists in the system or not, or whether an unused account was used, also requires a chain of events, as a single event cannot indicate that the system was attacked.

This program differs from monitoring systems in that monitoring systems analyze large amounts of data, while this program takes a massive amount of data and creates an initial system run focused specifically on identifying fake accounts. It looks at the account's usage and creation time and the IP of the person who used or created the account. This program serves as a support system. It's important to note that this system doesn't replace human resources, but rather serves as an assistant to automate certain aspects of the workflow.

This program doesn't scan a single log for fake accounts; instead, it builds an algorithm of events that could imply the creation of a fake account. This provides a more complete event diagram for the cybersecurity analyst. The monitoring systems themselves operate on a similar system, where the severity of an incident depends on the chain of events [35]. The chain of events can be used to determine the severity of a given incident. For example, the beginning may indicate the endpoint from which the account was created, the time it was created, and the IP address from which it was created. Furthermore, the groups to which the account was added within Active Directory also influences its severity. It also monitors the use of Kerberos attacks [42].

This program reads Windows Security logs in JSONL or JSON format. This format is widely accepted and readable by the system, so it was chosen as the program's reading method. Another reason for choosing JSON is that it is exportable and unique for different systems, and Python easily reads it. It can be used in various environments. Python was chosen because it easily scales if the program needs to be improved without changing the architecture [31].

When outputting results, this program produces three files: one HTML file. This is the most important of all the others, as it displays event cases in a human-readable format. This file allows you to visually review what happened and its severity. Then there's the CSV file format, which displays key events in a table. This is important if

you need to analyze the cases in a compressed format.

Another important feature of this program, as mentioned, is its HTML visualization (Figure 2.1; 2.2). The program not only evaluates an event but also provides a rationale for it. Rationale is one of the foundations of a good investigation. Rationale shows the chain of events that were used to determine whether an incident is dangerous or not. More precisely, detector-what-explains-events. This developed program not only classifies incidents but also provides the cause and accompanying information for a detailed understanding of the situation in order to visualize the situation as it exists in the system. The system displays when the process was created, the name or description of the process, Event ID, workstation, target and subject, user, host, and group. LogonType is also included, as it is also critical. The system also provides a reasoning for clarity. The system also provides reasoning to make it clear in the Details section.

создания/активации аккаунта (4/06) (+5) | Обнаружена ускоренная цепочка событий после создания/активации аккаунта (+15)

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType	Process	Attribute
2026-02-03T19:12:10+00:00	4720	User account created	SERVER2	suspectuser	random123abc456def							
2026-02-03T19:14:00+00:00	4722	User account enabled	SERVER2	suspectuser	random123abc456def							
2026-02-03T19:15:30+00:00	4728	Member added to security-enabled global group	DC01	suspectuser		Administrators	CN=random123abc456def,DC=FABRIKAM					
2026-02-03T19:16:05+00:00	4732	Member added to security-enabled local group	DC01	suspectuser		Domain Admins	CN=random123abc456def,DC=FABRIKAM					
2026-02-03T19:18:10+00:00	4625	Failed logon	DC01	-	random123abc456def			198.51.100.23	PC-UNKNOWN	3		
2026-02-03T19:19:35+00:00	4624	Successful logon	DC01	-	random123abc456def			198.51.100.23	PC-UNKNOWN	10		
2026-02-03T19:21:00+00:00	4768	Kerberos TGT requested	DC01		random123abc456def			198.51.100.23				
2026-02-03T19:21:40+00:00	4769	Kerberos service ticket requested	DC01		random123abc456def			198.51.100.23				
2026-02-04T12:02:10+00:00	4768	Kerberos TGT requested	DC01		random123abc456def			198.51.100.23				
2026-02-04T12:03:30+00:00	4769	Kerberos service ticket requested	DC01		random123abc456def			198.51.100.23				

Figure 2.1 - HTML file as summary of the case system

Details

random123abc456def — HIGH (95)

- **Case origin:** created
- **Case type:** suspicious_privileged_kerberos_account
- **Created / activated:** 2026-02-03T19:12:10+00:00
- **Creator:** suspectuser
- **Creator source:** IP=192.168.1.100; WS=ws2
- **Risk categories:** identity, kerberos, privilege, timing
- **Reasons:** Создание/активация аккаунта вне рабочих часов или в выходной день (+20) | Имя аккаунта похоже на автоматически сгенерированное: random123abc456def (+10) | Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) | Добавление в привилегированные группы: Administrators, Domain Admins (+35) | Kerberos TGT-запрос зафиксирован вскоре после создания/активации аккаунта (4768) (+5) | Обнаружена ускоренная цепочка событий после создания/активации аккаунта (+15)

Figure 2.2 - HTML file as summary of the case system

As was evident, the system's primary effectiveness is strictly dependent on its base. The primary base for detection in this case is the foundation, and one of these foundations is the Event ID. Let's pay attention to Table 2.1.

Table 2.1 - Description of Event ID numbers

Event ID	Description
4720	Describes when the user was created in the system.
4722	Describes when the user was activated in the system.
4723	Describes when there was an attempt to change the account password in

	the system.
4724	Describes when the system attempted to reset the account password.
4725	Describes when a user was deactivated in the system.
4726	Describes when a user was deleted from the system.
4728	Describes when a user or account was added to the security-enabled global group in the system
4672	Describes when new users within the system are given special privileges when logging in.
4768	Describes when the system was requested to authenticate a Kerberos TGT ticket.
4769	Describes when the Kerberos TGT ticket service was requested from the system.
4771	Describes when the system's Kerberos pre-authentication failed.
4776	Describes when the Domain Controller attempted to validate the account data.
5136	Describes when a directory service object was modified in the system.

As the table shows, the creation of fake accounts involves a huge number of processes. This all reveals the chain of events underlying the very idea of creating a fake account. If the program only recorded account creation, the report would be incomplete, which could yield incorrect data. Accounts are created and deleted almost constantly in the Active Directory system. But since the program's system includes a complete list of triggers for detecting fake accounts, a whole network of chains can be seen. An attacker can exploit vulnerabilities in the Kerberos network, Mimikatz, and other hacking tools, and the operating pattern of these tools is virtually the same: exploiting obvious vulnerabilities that can allow the hacker to create an account [42, 43].

The program system first creates a case. The entry point for a case is the creation of an account or the activation of an old account that was forgotten within the Active Directory system. The entire case is built around this incident and takes into account all events within the system to avoid straying into another axiom. Events refer to the rules and policies that were used, when the account was created, by whom the account was created, the reason for its creation, the host from which it was created, whether the password was changed or reset, whether a Kerberos ticket was requested, whether a group policy or group was used, and what actions were performed afterward. All these processes are built around a single case. Next, the severity level for this case is given (Figure 2.3).

Account	Case Origin	Case Type	Risk	Score	Created / Activated	Creator	Creator IP	Creator WS	Categories	Reasons
random123abc456def	created	suspicious_privileged_kerberos_account	HIGH	95	2026-02-03T19:12:10+00:00	suspectuser	192.168.1.100	ws2	identity, kerberos, privilege, timing	Создание/активация аккаунта вне рабочих часов или в выходной день (+20) Имя аккаунта похоже на автоматически сгенерированное: random123abc456def (+10) Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) Добавление в привилегированные группы: Administrators, Domain Admins (+35) Kerberos TGT-запрос зафиксирован вскоре после создания/активации аккаунта (4768) (+5) Обнаружена ускоренная цепочка событий после создания/активации аккаунта (+15)
svc_backup	created	suspicious_privileged_kerberos_account	HIGH	90	2026-02-03T19:30:00+00:00	itadmin	10.0.0.50	jump01	identity, kerberos, privilege, timing	Имя аккаунта выглядит типовым: svc_backup (-5) Создание/активация аккаунта вне рабочих часов или в выходной день (+20) Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) Добавление в привилегированные группы: Backup Operators (+35) Вскоре после создания/активации зафиксирована попытка смены/сброса пароля (+10) Kerberos TGT-запрос зафиксирован вскоре после создания/активации аккаунта (4768) (+5) Обнаружена ускоренная цепочка событий после создания/активации аккаунта (+15)
admin	created	suspicious_privileged_account	HIGH	75	2026-02-03T18:52:10+00:00	normaluser	203.0.113.50	ws-ext01	identity, privilege, timing	Создание/активация аккаунта вне рабочих часов или в выходной день (+20) Имя аккаунта похоже на служебное/привилегированное: admin (+10) Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) Добавление в привилегированные группы: Domain Admins, Enterprise Admins (+35)
guest	created	suspicious_privileged_account	HIGH	75	2026-02-04T06:13:25+00:00	guest	10.2.3.4	kiosk1	identity, privilege, timing	Создание/активация аккаунта вне рабочих часов или в выходной день (+20) Имя аккаунта похоже на служебное/привилегированное: guest (+10) Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) Добавление в привилегированные группы: Schema Admins (+35)
support	created	suspicious_privileged_account	HIGH	75	2026-02-03T20:06:15+00:00	helpdesk	10.0.1.12	ws3	identity, privilege, timing	Создание/активация аккаунта вне рабочих часов или в выходной день (+20) Имя аккаунта похоже на служебное/привилегированное: support (+10) Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) Добавление в привилегированные группы: Enterprise Admins (+35)
testuser	created	suspicious_privileged_account	HIGH	70	2026-02-03T21:12:30+00:00	guest	10.2.3.4	kiosk1	identity, privilege, timing	Имя аккаунта выглядит типовым: testuser (-5) Создание/активация аккаунта вне рабочих часов или в выходной день (+20) Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) Добавление в привилегированные группы: Schema Admins (+35) Вскоре после создания/активации зафиксирована попытка смены/сброса пароля (+10)

Figure 2.3 - Summary of every case type

In addition to the risk level, the event is also designated as "timing-related," which means the incident occurred at a suspicious time. "Privilege-related," which indicates that the account has ascended through the ranks, gaining higher privileges. Based on this information, a brief description of the incident is created as a case-type (Fig.2.4).

guest — HIGH (75)

- **Case origin:** created
- **Case type:** suspicious_privileged_account
- **Created / activated:** 2026-02-04T06:13:25+00:00
- **Creator:** guest
- **Creator source:** IP=10.2.3.4; WS=kiosk1
- **Risk categories:** identity, privilege, timing
- **Reasons:** Создание/активация аккаунта вне рабочих часов или в выходной день (+20) | Имя аккаунта похоже на служебное/привилегированное: guest (+10) | Аккаунт включён вскоре после создания/обнаружения (<=15 минут) (+10) | Добавление в привилегированные группы: Schema Admins (+35)

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType	Process	Attribute
2026-02-04T06:13:25+00:00	4720	User account created	DC01	guest	guest							
2026-02-04T06:20:05+00:00	4722	User account enabled	DC01	guest	guest							
2026-02-04T06:22:40+00:00	4728	Member added to security-enabled global group	DC01	admin		Schema Admins	CN=guest,DC=CONTOSO					
2026-02-04T06:25:00+00:00	4624	Successful logon	DC01	-	guest			203.0.113.50	WS-EXT01	10		

Figure 2.4 - Detailed description of the case

The program's system also checks the list of allowed users who have permission to create an account, which is crucial for detecting fake accounts. Using a special script function, you can immediately display which names are allowed to filter out legitimate accounts. The flag for this function is "allow-list."

The next function is privilege access analysis. The script monitors how quickly higher privileges were gained. This is important in the system, as the combination of account creation and subsequent privilege escalation is critical for detecting fake accounts. Kerberos ticket requests are also critical [44, 45].

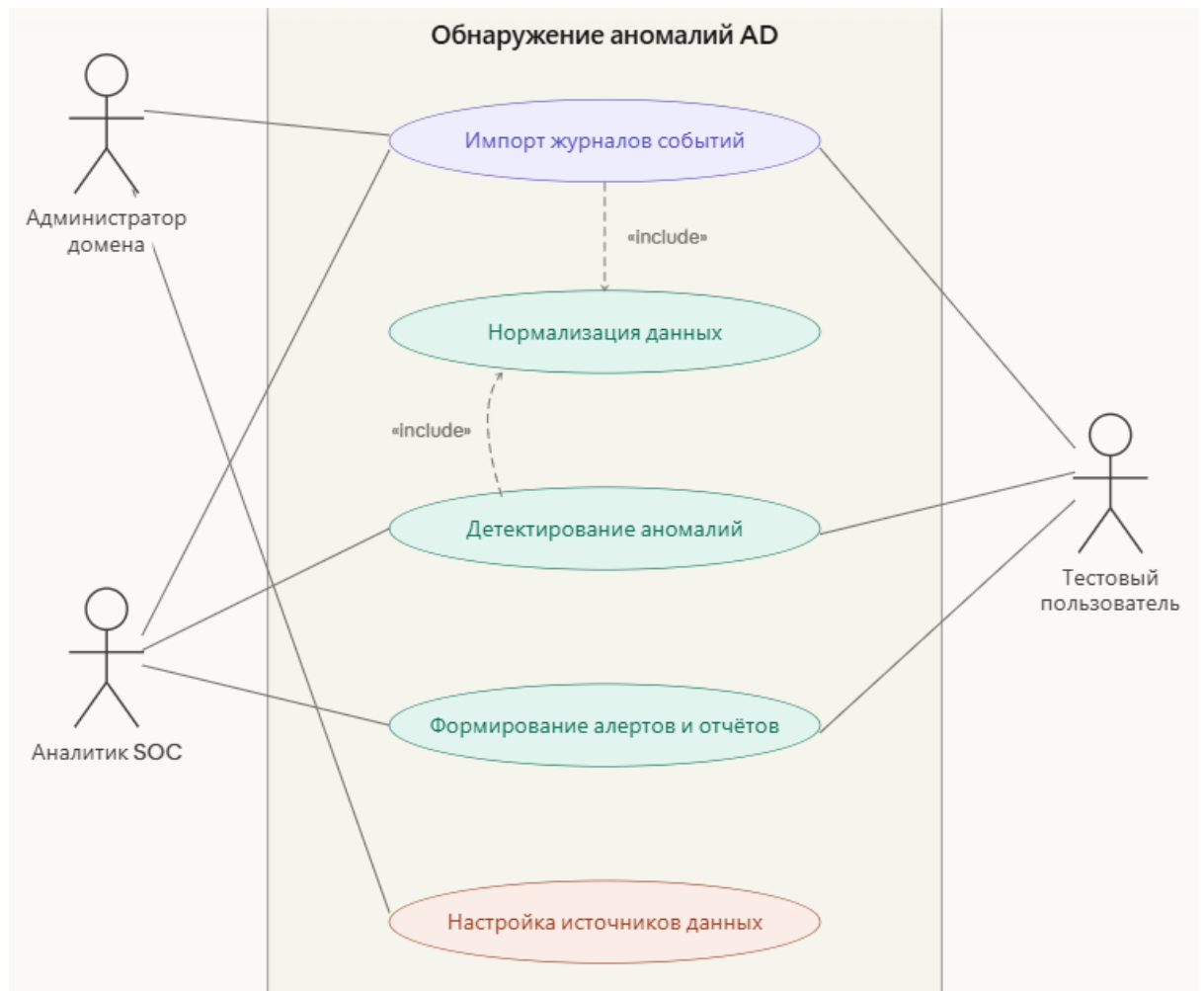


Figure 2.5 – Kerberos Authentication Flow in Active Directory

2.2 Program Code Explanation

In this section, some basic structure of this program will be explained.

```

EVENT_DESCR = {
    4624: "Successful logon",
    4625: "Failed logon",
    4648: "Logon attempted using explicit credentials",
    4672: "Special privileges assigned to new logon",
    4688: "Process creation",
    4720: "User account created",
    4722: "User account enabled",
    4723: "Attempt to change account password",
    4724: "Attempt to reset account password",
    4725: "User account disabled",
    4726: "User account deleted",
    4728: "Member added to security-enabled global group",
    4732: "Member added to security-enabled local group",

```

```

4738: "User account was changed",
4740: "User account locked out",
4756: "Member added to security-enabled universal group",
4768: "Kerberos TGT requested",
4769: "Kerberos service ticket requested",
4771: "Kerberos pre-authentication failed",
4776: "NTLM authentication attempted",
5136: "Directory Service object modified",
}

```

This code block is important because it provides reference information without the need to go online to search for it. As mentioned, Windows Security Logs works with Event IDs. This structure is logically simpler, as it stores log information digitally, providing access to a wide range of information within a directory on the Microsoft website or related to cybersecurity [7, 9, 14]. Cybersecurity analysts constantly need to log in to the network or have an entire directory at hand to determine the contents of Event ID data. However, with this block within the program, this work is delegated to the program itself. For this reason, this block was included. This block also performs the task of interpreting JSONL or JSON arrays in the correct order. The correct order is important in this case, as the hacking structure itself occurs in such a way that the fake account creation scheme is practically identical in functionality. Therefore, this code block contains a database to which the code structure itself will refer. The semantic, functional, and architectural parts of this code block are tightly coupled with other code blocks, as the other blocks are crucial within the program. This block functions as a database for determining the incident algorithm to determine whether an attack occurred or not, namely, whether a legitimate account was created within the system. `SUPPORTED_EVENT_IDS = set(EVENT_DESCR.keys())` This part of the code block compares Event IDs from the `EVENT_DESCR` block. The importance of this block lies in its function as a comparison part of the JSON, which is critical for the program to detect fake account creation. This block may seem ordinary, but its significance lies in the fact that it separates noise from real data. The list of Event IDs is vast, and they identify various types of processes, from turning on a PC to downloading a game from a pirated website. It also includes the download of a Word document. These are just a few examples of what an Event ID entails. In this case, without this block of code `SUPPORTED_EVENT_IDS = set(EVENT_DESCR.keys())` it would be difficult for the program to determine and compare real data with noise [46].

```

PRIV_GROUP_DEFAULT = {
    "domain admins",
    "enterprise admins",
    "administrators",
    "account operators",
    "backup operators",
}

```

```

    "schema admins",
    "server operators",
    "print operators",
    "dnsadmins",
}

```

This code block contains the names of privileged groups within the Active Directory ecosystem. There may be more or fewer such groups, but this code block demonstrates the criticality of these groups, as when creating a fake account, an attacker will attempt to integrate the account into a highly privileged environment [8, 41]. This is critical because when creating an Active Directory environment, administrators create different groups within the internal user ecosystem. Some groups may have administrator-level privileges, while others are not members of these groups, and therefore, users without privileges are assigned to groups without privileges. This code block is linked to other code blocks to detect compromises using analysis and a parser read via a JSON file. This code block can also be modified for different situations, as this code block only shows basic groups with privileges. However, each company's Active Directory environment may be different, and they may contain different or entirely different user groups.

```

DEFAULT_WEIGHTS = {
    # positive risk
    "offhours": 20,
    "creator_not_allowed": 15,
    "creator_host_not_allowed": 10,
    "susp_name": 10,
    "rand_name": 10,
    "enabled_soon": 10,
    "priv_group": 35,
    "special_priv": 20,
    "first_logon_ext_ip": 15,
    "many_fails": 10,
    "explicit_creds": 10,
    "pwd_reset_soon": 10,
    "acct_changed_soon": 10,
    "acct_disabled_soon": 5,
    "acct_deleted_soon": 15,
    "lockout_after_create": 10,
    "kerberos_tgt_soon": 5,
    "kerberos_tgs_burst": 10,
    "kerberos_failures": 10,
    "ntlm_attempts": 5,
    "ds_object_changed": 10,
    "burst_creator": 20,
    "activity_chain": 15,
}

```

```

    "enabled_without_create": 15,

    # negative / normalization
    "creator_allowed_bonus": -10,
    "creator_host_allowed_bonus": -5,
    "business_hours_bonus": -5,
    "corp_ip_bonus": -5,
    "normal_name_bonus": -5,
}

```

This section of the code block is also critically important because it evaluates event cases to determine the threat level. Threat level is a system that exists everywhere. SIEM tools for SOC analysts, such as Qradar, Splunk, and others, have a risk assessment system based on policies, rules, and an internal database that enables the software system to respond to critical situations. This code block performs the same function as those tools, but in a more focused manner, focusing on detecting fake accounts [33, 35]. As can be seen from the code block, it has two areas. One area is focused on increasing the risk level, while the other, conversely, reduces it. As shown in the code block, one of the most critical moments is adding an account to the administrator group. For this reason, it has the highest criticality score [39, 41]. Creating an account outside of normal working hours is also critical, as basic practice shows that an account created outside of normal working hours has a high risk of being a fake account. One may wonder if unexpected situations occur when an account is created outside of normal working hours. But in practice, cybersecurity specialists have the opportunity to clarify such questions with the host from which the fake account was created [42, 45].

```

def _try_int(x: Any) -> Optional[int]:
    try:
        return int(x)
    except Exception:
        return None

def norm_str(x: Any) -> Optional[str]:
    if x is None:
        return None
    s = str(x).strip()
    return s if s else None

```

Despite the simplicity of this code, it is crucial to the overall program architecture. When converting logs from the EVTX format, which stores system-wide log data, problems can arise, such as extra spaces or indents within the original file that could impair system operation. This is where this code block comes in handy, searching for such cracks within the JSON to identify such errors. `_try_int` correctly organizes the JSON data array, and the `norm_str` block removes unnecessary characters such as extra spaces. Without this code block, the system could crash early. If errors are

detected, the system returns None as a result, and the program continues running [31].

Block of code:

```
# Helpers  
#
```

```
def _try_int(x: Any) -> Optional[int]:  
    try:  
        return int(x)  
    except Exception:  
        return None
```

```
def norm_str(x: Any) -> Optional[str]:  
    if x is None:  
        return None  
    s = str(x).strip()  
    return s if s else None
```

```
def get_path(d: Dict[str, Any], *paths: str, default=None):  
    for p in paths:  
        cur = d  
        ok = True  
        for part in p.split("."):   
            if isinstance(cur, dict) and part in cur:  
                cur = cur[part]  
            else:  
                ok = False  
                break  
        if ok:  
            return cur  
    return default
```

```
def parse_ts(evt: Dict[str, Any]) -> Optional[datetime]:  
    val = get_path(  
        evt,  
        "@timestamp",  
        "timestamp",  
        "TimeCreated",  
        "time_created",  
        "winlog.time_created",  
        "event.created",  
        "event.ingested",  
    )  
    if val is None:
```

```

    return None

if isinstance(val, datetime):
    return val if val.tzinfo else val.replace(tzinfo=timezone.utc)

if isinstance(val, (int, float)):
    try:
        return datetime.fromtimestamp(float(val), tz=timezone.utc)
    except Exception:
        return None

if isinstance(val, str):
    s = val.strip()
    if s.endswith("Z"):
        s = s[:-1] + "+00:00"
    try:
        dt = datetime.fromisoformat(s)
        return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)
    except Exception:
        pass

    for fmt in ("%Y-%m-%d %H:%M:%S", "%Y-%m-%d %H:%M:%S.%f"):
        try:
            return datetime.strptime(s, fmt).replace(tzinfo=timezone.utc)
        except Exception:
            continue
    return None

def is_weekend(dt: datetime) -> bool:
    return dt.weekday() >= 5

def outside_business_hours(dt: datetime, start_hour: int, end_hour: int) -> bool:
    h = dt.hour
    if start_hour < end_hour:
        return not (start_hour <= h < end_hour)
    return (end_hour <= h < start_hour)

def looks_random(name: str) -> bool:
    if not name:
        return False
    n = name.strip()
    if n.endswith("$"):
        # machine account
        return False

```

```

if len(n) >= 14:
    has_alpha = any(c.isalpha() for c in n)
    has_digit = any(c.isdigit() for c in n)
    has_sep = any(c in "._- " for c in n)
    return has_alpha and has_digit and not has_sep
return False

def normalize_group(g: Optional[str]) -> Optional[str]:
    return g.strip().lower() if g else None

def normalize_identity(x: Optional[str]) -> Optional[str]:
    """
    Normalize user / member identity:
    - strips domain prefix DOMAIN\\user
    - strips UPN user@domain if present
    - extracts CN=user from DN if present
    - lowercases
    """
    if not x:
        return None
    s = x.strip()

    # DN like CN=user1,OU=...
    m = re.search(r"(?i)\\bCN=([^\,]+)", s)
    if m:
        s = m.group(1).strip()

    # DOMAIN\\user
    if "\\\" in s:
        s = s.split("\\")[1]

    # UPN user@domain
    if "@" in s:
        s = s.split("@")[0]

    return s.strip().lower() if s.strip() else None

def member_matches_account(member_name: Optional[str], account: str) ->
bool:
    if not member_name or not account:
        return False
    m = normalize_identity(member_name)
    a = normalize_identity(account)
    return bool(m and a and m == a)

```

```

def parse_cidrs(cidrs: List[str]) -> List[ipaddress._BaseNetwork]:
    nets: List[ipaddress._BaseNetwork] = []
    for c in cidrs:
        try:
            nets.append(ipaddress.ip_network(c.strip(), strict=False))
        except Exception:
            pass
    return nets

def in_any_cidr(ip: str, nets: List[ipaddress._BaseNetwork]) -> bool:
    try:
        addr = ipaddress.ip_address(ip)
        return any(addr in net for net in nets)
    except Exception:
        return False

def load_list_file(path: Optional[str]) -> Optional[set]:
    if not path:
        return None
    p = Path(path)
    if not p.exists():
        return None
    out = set()
    for line in p.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if line and not line.startswith("#"):
            out.add(line)
    return out

def lc_set(xs: Optional[Set[str]]) -> Set[str]:
    if not xs:
        return set()
    return {str(x).strip().lower() for x in xs if str(x).strip()}

def safe(s: Any) -> str:
    return html.escape(str(s)) if s is not None else ""

def fmt_dt(dt: datetime) -> str:
    return dt.isoformat()

```

#

This section of the script is helpful because it allows the code to detect a JSON file. How exactly does it work? As mentioned earlier, a JSON file is a formatted log

file format for Windows Security Logs. This section helps those who need to make the code more readable, as when converting one file format to another, and in our case, to JSON, errors can occur. For example, an extra space, an extra period, an extra comma, unnecessary information that's completely unnecessary. This section of the code helps by making the code more readable—a readable format for this particular script. This section of the code can be considered the most fundamental, as without this section, the rest of the program itself won't work at all. For this reason, it's important to pay close attention to this section of the script.

3 PRACTICAL PART DETAILED PROJECT DEVELOPMENT

3.1 The practical visualization of how Program works Part 1

For this project's practical application, two different JSON-formatted logs were selected, each representing a typical corporate log file. This project's program script is primarily focused on identifying patterns and creating logs based on standard Active Directory operation patterns. This means that if all the logs were identical for attacks and fake account creation, the system operation of the project program could not be considered correct. Correct operation must demonstrate the ability of the program system to identify fake account patterns based on the database included in the program script. It's also worth noting from the previous topic that this script is designed to create cases based on account names to identify patterns and operational telemetry.

Below is an example of a converted EVTX log format into JSON. This is one of the first scripts used.

```
{"@timestamp":"2026-02-03T18:40:00.000000+00:00","winlog":{"event_id":4624,"channel":"Security","computer_name":"DC01","record_id":"900001","event_data":{"SubjectUserName":"-","SubjectDomainName":"-","SubjectLogonId":"0x0","TargetUserName":"normaluser","TargetDomainName":"CONTOSO","TargetLogonId":"0x4bde8a","IpAddress":"203.0.113.50","WorkstationName":"WS-EXT01","LogonType":"10"}}}

{"@timestamp":"2026-02-03T18:41:05.000000+00:00","winlog":{"event_id":4688,"channel":"Security","computer_name":"DC01","record_id":"900002","event_data":{"SubjectUserName":"normaluser","SubjectDomainName":"CONTOSO","SubjectLogonId":"0x4bde8a","NewProcessName":"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe"}}}

{"@timestamp":"2026-02-03T18:44:10.000000+00:00","winlog":{"event_id":4624,"channel":"Security","computer_name":"SERVER2","record_id":"900003","event_data":{"SubjectUserName":"-","SubjectDomainName":"-","SubjectLogonId":"0x0","TargetUserName":"suspectuser","TargetDomainName":"ADATUM","TargetLogonId":"0x63310a","IpAddress":"192.168.1.100","WorkstationName":"WS2","LogonType":"10"}}}

{"@timestamp":"2026-02-03T18:45:20.000000+00:00","winlog":{"event_id":4688,"channel":"Security","computer_name":"SERVER2","record_id":"900004","event_data":{"SubjectUserName":"suspectuser","SubjectDomainName":"ADATUM","SubjectLogonId":"0x63310a","NewProcessName":"C:\\Windows\\System32\\cmd.exe"}}}

{"@timestamp":"2026-02-03T18:47:00.000000+00:00","winlog":{"event_id":4624,"channel":"Security","computer_name":"DC01","record_id":"900005","event_data":{"SubjectUserName":"-","SubjectDomainName":"-","SubjectLogonId":"0x0","TargetUserName":"itadmin","TargetDomainName":"CONTOSO","TargetLogonId":"0x9a011f","IpAddress":"10.0.0.50","WorkstationName":"JUMP01","LogonType":"3"}}}

{"@timestamp":"2026-02-03T18:48:12.000000+00:00","winlog":{"event_id":4688,"channel":"Security","computer_name":"DC01","record_id":"900006","event_data":{"SubjectUserName":"itadmin","SubjectDomainName":"CONTOSO","SubjectLogonId":"0x9a011f","NewProcessName":"C:\\Windows\\System32\\net.exe"}}}
```

```

{"@timestamp":"2026-02-03T18:52:10.000000+00:00","winlog":{"event_id":4720,"channel":"Security","computer_name":"DC01","record_id":"900007","event_data":{"SubjectUserName":"normaluser","SubjectDomainName":"CONTOSO","SubjectLogonId":"0x4bde8a","TargetUserName":"admin","TargetDomainName":"CONTOSO","TargetSid":"S-1-5-21-1111111111-2222"}}}
{"@timestamp":"2026-02-03T18:58:20.000000+00:00","winlog":{"event_id":4722,"channel":"Security","computer_name":"DC01","record_id":"900008","event_data":{"SubjectUserName":"normaluser","SubjectDomainName":"CONTOSO","SubjectLogonId":"0x4bde8a","TargetUserName":"admin","TargetDomainName":"CONTOSO","TargetSid":"S-1-5-21-1111111111-2222"}}}

```

In the first JSON log, let's call it event1. This is typical behavior when IP addresses are unknown. It often happens in a system that IP addresses are unknown or that they are system IP addresses that only show what happened, as there are many tools that hide the very existence of IP addresses, or hackers themselves can delete log data if they know they won't be returned.

AD Fake Account Detection Report

Generated: 2026-04-07T16:28:24.469059+00:00

Total parsed events: 192

Total suspicious findings: 3

Table 3.1 – Overview

Metric	Value
Findings: HIGH	0
Findings: MEDIUM	3
Findings: LOW	0
Findings: INFO	0

Table 3.2 – Case type distribution

Case type	Count
suspicious_privileged_account	2
suspicious_new_or_enabled_account	1

Table 3.3 – Risk category distribution

Category	Count
identity	3
timing	3
privilege	2

Table 3.4 – Most frequent triggers

Trigger	Count
Account creation/activation outside business hours or on a weekend	3
Adding to Privileged Groups: Enterprise Admins	2
The account name is similar to the service/privileged account: root	1
The account name looks typical: user1	1
The account name looks typical: suspectuser	1

An account activation was detected without an observable creation event.	1
The account is enabled shortly after creation/discovery	1

Table 3.5 – Summary

Account	Case Origin	Case Type	Risk	Score	Created / Activated	Creator	Categories	Reasons
root	created	suspicious_privileged_account	MEDIUM	65	2026-02-06T06:10:07.623720+00:00	normaluser	identity, privilege, timing	Account creation/activation outside of business hours or on a weekend (+20) Account name similar to service/privileged account: root (+10) Addition to privileged groups: Enterprise Admins (+35)
user1	created	suspicious_privileged_account	MEDIUM	50	2026-02-08T21:22:10.094605+00:00	support	identity, privilege, timing	The account name looks typical: user1 (-5) Account creation/activation outside of business hours or on a

								weekend (+20) Addition to privileged groups: Enterprise Admins (+35)
suspectuser	enabled_only	suspicious_new_or_enabled_account	MEDIUM	40	2026-02-03T21:53:15.977759+00:00	random123abc456def	identity, timing	The account name appears to be typical: suspectuser (-5) Account creation/activation outside of business hours or on a weekend (+20) Account activation recorded without an observable creation event (+15) Account enabled shortly after creation/detection (≤ 15 minutes) (+10)

Details
root — MEDIUM (65)

Case origin: created
Case type: suspicious_privileged_account
Created / activated: 2026-02-06T06:10:07.623720+00:00
Creator: normaluser
Creator source: IP=; WS=
Risk categories: identity, privilege, timing
Reasons: Account creation/activation outside of business hours or on a weekend
(+20) | Account name similar to a service/privileged account: root (+10)
| Adding to privileged groups: Enterprise Admins (+35)

Table 3.6 – Practical part

Time	Event ID	Description	Host	SubjectUser	TargetUser	Group	Member
2026-02-06 06:10:07	4720	User account created	WORKSTATION1	normaluser	root	—	—
2026-02-06 08:44:52	4720	User account created	WORKSTATION1	root	root	—	—
2026-02-07 03:46:02	4732	Member added to security-enabled local group	DC01	admin	—	Enterprise Admins	CN=root,DC=ADATUM

user1 — MEDIUM (50)

Case origin: created
Case type: suspicious_privileged_account
Created / activated: 2026-02-08T21:22:10.094605+00:00
Creator: support
Creator source: IP=; WS=
Risk categories: identity, privilege, timing

Table 3.7 – Practical part

Time	Event ID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-08 21:22:10	4720	User account created	WORKSTATION1	support	user1	—	—	—	—	—
2026-02-	4732	Member added	WORKSTATION1	suspectuser	—	Enterprise	CN=user1,DC=FABRIKAM	—	—	—

09 02:3 2:57		d to secur ity- enabl ed local group				Adm ins				
202 6- 02- 09 05:1 2:26	472 0	User accou nt creat ed	SERVER 2	root	user1	—	—	—	—	—
202 6- 02- 09 06:2 8:04	472 0	User accou nt creat ed	SERVER 2	root	user1	—	—	—	—	—
202 6- 02- 09 13:3 1:46	462 5	Faile d logon	WORKS TATION 1	-	user1	—	—	192.16 8.1.10 0	WS2	3

Reasons: Account name appears to be typical: user1 (-5) | Account creation/activation outside of business hours or on a weekend (+20) | Addition to privileged groups: Enterprise Admins (+35)

suspectuser — MEDIUM (40)

Case origin: enabled_only

Case type: suspicious_new_or_enabled_account

Created / activated: 2026-02-03T21:53:15.977759+00:00

Creator: random123abc456def

Creator source: IP=; WS=

Risk categories: identity, timing

Reasons: The account name appears typical: suspectuser (-5) | Account creation/activation outside of business hours or on a weekend (+20) |

Account activation detected without an observable creation event (+15) | Account enabled shortly after creation/detection (<=15 minutes) (+10)

Table 3.8 – Practical part

Time	EventID	Description	Host	SubjectUser	TargetUser
2026-02-03 21:53:15	4722	User account enabled	DC01	random123abc456def	suspectuser
2026-02-04 03:48:55	4722	User account enabled	DC01	user1	suspectuser

It was also mentioned that the program report also has a JSON file of findings.

This one provides the full case architecture. Below, a portion of the findings are shown:

```
{
  "account": "root",
  "created_at": "2026-02-06T06:10:07.623720+00:00",
  "case_origin": "created",
  "case_type": "suspicious_privileged_account",
  "risk_score": 65,
  "risk_level": "MEDIUM",
  "risk_categories": [
    "identity",
    "privilege",
    "timing"
  ],
  "creator": "normaluser",
  "creator_ip": null,
  "creator_workstation": null,
  "reasons": [
    "Account creation/activation outside business hours or on a weekend (+20)",
    "The account name is similar to the service/privileged account: root (+10)",
    "Adding to privileged groups: Enterprise Admins (+35)"
  ],
}
```

3.2 Practical Visualization of How the Program Works Part 2

The second JSON file already contains dangerous data, including IP addresses. 87 events were processed, with an average of 7 cases.

Here is some of the data from these events:

```
{"@timestamp": "2026-02-14T10:20:28.746801+00:00", "winlog": {"event_id": 4768, "channel": "Security", "computer_name": "DC01", "record_id": "163453", "event_data": {"TargetUserName": "admin", "TargetDomainName": "ADATUM", "IpAddress": "10.0.0.50"}}}
{"@timestamp": "2026-02-14T10:46:50.988363+00:00", "winlog": {"event_id": 4625, "channel": "Security", "computer_name": "WORKSTATION1", "record_id": "474005", "event_data": {"SubjectUserName": "-", "SubjectDomainName": "-", "SubjectLogonId": "0x0", "TargetUserName": "user1", "TargetDomainName": "CONTOSO", "TargetLogonId": "0xb070f5", "IpAddress": "172.16.0.10", "WorkstationName": "WS1", "LogonType": "10"}}}
{"@timestamp": "2026-02-14T12:24:38.560085+00:00", "winlog": {"event_id": 4672, "channel": "Security", "computer_name": "DC01", "record_id": "439406", "event_data": {"SubjectUserName": "normaluser", "SubjectDomainName": "CONTOSO", "SubjectLogonId": "0xe333f8"}}}
{"@timestamp": "2026-02-14T14:06:14.048652+00:00", "winlog": {"event_id": 4672, "channel": "Security", "computer_name": "SERVER2", "record_id": "577488", "event_data": {"SubjectUserName": "normaluser", "SubjectDomainName": "ADATUM", "SubjectLogonId": "0xaff773"}}}
{"@timestamp": "2026-02-14T14:33:41.725203+00:00", "winlog": {"event_id": 4688, "channel": "Security", "computer_name": "SERVER2", "record_id": "863939", "event_data":
```

```

{"SubjectUserName": "svc_backup", "SubjectDomainName": "ADATUM", "SubjectLogonId":
"0x7eba47", "NewProcessName": "C:\\Windows\\System32\\lsass.exe"}}}
{"@timestamp": "2026-02-14T19:40:08.125518+00:00", "winlog": {"event_id": 4672,
"channel": "Security", "computer_name": "DC01", "record_id": "871191", "event_data":
{"SubjectUserName": "suspectuser", "SubjectDomainName": "FABRIKAM", "SubjectLogonId":
"0xe16b08"}}}
{"@timestamp": "2026-02-14T20:32:20.336413+00:00", "winlog": {"event_id": 4624,
"channel": "Security", "computer_name": "WORKSTATION1", "record_id": "574112",
"event_data": {"SubjectUserName": "-", "SubjectDomainName": "-", "SubjectLogonId": "0x0",
"TargetUserName": "admin", "TargetDomainName": "FABRIKAM", "TargetLogonId": "0x6ce70f",
"IpAddress": "192.168.1.100", "WorkstationName": "WS2", "LogonType": "10"}}}
{"@timestamp": "2026-02-14T20:40:41.310516+00:00", "winlog": {"event_id": 4720,
"channel": "Security", "computer_name": "WORKSTATION1", "record_id": "986116",
"event_data": {"SubjectUserName": "testuser", "SubjectDomainName": "CONTOSO",
"SubjectLogonId": "0x6a4c66", "TargetUserName": "testuser", "TargetDomainName":
"CONTOSO", "TargetSid": "S-1-5-21-3190006295-7433"}}}
{"@timestamp": "2026-02-14T22:16:51.679789+00:00", "winlog": {"event_id": 4768,
"channel": "Security", "computer_name": "DC01", "record_id": "661169", "event_data":
{"TargetUserName": "svc_backup", "TargetDomainName": "FABRIKAM", "IpAddress":
"192.168.1.100"}}}
{"@timestamp": "2026-02-15T01:37:14.812663+00:00", "winlog": {"event_id": 4768,
"channel": "Security", "computer_name": "WORKSTATION1", "record_id": "945232",
"event_data": {"TargetUserName": "admin", "TargetDomainName": "CONTOSO", "IpAddress":
"172.16.0.10"}}}

```

The Result of the logs AD Fake Account Detection Report
 Generated: 2026-04-07T16:33:29.471085+00:00
 Total parsed events: 84
 Total suspicious findings: 7

Table 3.9 – Overview

Metric	Value
Findings: HIGH	6
Findings: MEDIUM	1
Findings: LOW	0
Findings: INFO	0

Table 3.10 – Case type distribution

Case type	Count
suspicious privileged account	5
suspicious privileged kerberos account	2

Table 3.11 – Risk category distribution

Category	Count
identity	7
privilege	7
timing	7
kerberos	2

Table 3.12 – Most frequent triggers

Trigger	Count
The account is enabled shortly after creation/discovery	7
Account creation/activation outside business hours or on a weekend	6
Kerberos TGT request recorded shortly after account creation/activation	2
Accelerated chain of events detected after account creation/activation	2
An attempt to change/reset the password was recorded shortly after creation/activation	2
Adding to privileged groups: Schema Admins	2
Adding to Privileged Groups: Enterprise Admins	2
The account name appears to be automatically generated: random123abc456def	1
Adding to privileged groups: Administrators, Domain Admins	1
The account name looks typical: svc_backup	1
Adding to privileged groups: Backup Operators	1
The account name appears to be a service/privileged account: admin	1
Adding to privileged groups: Domain Admins, Enterprise Admins	1
The account name appears to be a service/privileged account: guest	1
The account name appears to be a service/privileged account: support	1

Table 3.13 – Summary

Account	Case Origin	Case Type	Risk	Score	Created / Activated	Creator	Creator IP	Creator WS	Categories	Reasons
---------	-------------	-----------	------	-------	---------------------	---------	------------	------------	------------	---------

random123 abc456def	cre ate d	suspicious_privileged _kerberos_account	HI G H	95	2026-02- 03T19:12: 10+00:00	suspe ctuser	192.16 8.1.100	ws2	identi ty, kerbe ros, privil ege, timin g	Account creation/act ivation outside of business hours or on a weekend (+20) Account name similar to auto- generated: random123 abc456def (+10) Account enabled shortly after creation/dis covery (<=15 minutes) (+10) Added to privileged groups: Administra tors, Domain Admins (+35) Kerberos TGT request detected shortly after account creation/act ivation (4768) (+5) Accelerate d chain of events detected after account creation/act
------------------------	-----------------	--	--------------	----	-----------------------------------	-----------------	-------------------	-----	--	---

										ivation (+15)
svc_backup	created	suspicious_privileged_kerberos_account	HIGH	90	2026-02-03T19:30:00+00:00	itadmin	10.0.0.50	jumper01	identity, kerberos, privilege, timing	The account name appears to be typical: svc_backup (-5) Account creation/activation outside of business hours or on a weekend (+20) Account enabled shortly after creation/discovery (<=15 minutes) (+10) Added to privileged groups: Backup Operators (+35) An attempt to change/reset a password was detected shortly

										after creation/activation (+10) A Kerberos TGT request was detected shortly after account creation/activation (4768) (+5) An accelerated chain of events was detected after account creation/activation (+15)
--	--	--	--	--	--	--	--	--	--	---

Details

random123abc456def — HIGH (95)

Case origin: created

Case type: suspicious_privileged_kerberos_account

Created / activated: 2026-02-03T19:12:10+00:00

Creator: suspectuser

Creator source: IP=192.168.1.100; WS=ws2

Risk categories: identity, kerberos, privilege, timing

Reasons: Account creation/activation outside of business hours or on a weekend (+20) | Account name similar to the one automatically generated:

Table 3.14 – Practical part

Time	Event ID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-03 19:12:10	4720	User account created	SERVER2	suspectuser	random123abc456def	—	—	—	—	—
2026-	4722	User acco	SERVE	suspectus	random123abc45	—	—	—	—	—

02-03 19:14:00		unt enab led	R2	er	6def					
2026-02-03 19:15:30	47 28	Me mbe r adde d to secu rity-enab led glob al grou p	DC 01	susp ectus er	—	Admi nistrat ors	CN=random123abc 456def,DC=FABR IKAM	—	—	—
2026-02-03 19:16:05	47 32	Me mbe r adde d to secu rity-enab led local grou p	DC 01	susp ectus er	—	Doma in Admi ns	CN=random123abc 456def,DC=FABR IKAM	—	—	—
2026-02-03 19:18:10	46 25	Faile d logo n	DC 01	—	random1 23abc45 6def	—	—	198.5 1.100. 23	PC- UNK NO WN	3
2026-02-03 19:19:35	46 24	Succ essful logo n	DC 01	—	random1 23abc45 6def	—	—	198.5 1.100. 23	PC- UNK NO WN	10
2026-02-03 19:21:00	47 68	Kerb eros TGT requ este d	DC 01	—	random1 23abc45 6def	—	—	198.5 1.100. 23	—	—

202 6- 02- 03 19: 21: 40	47 69	Kerb eros servi ce ticke t requ este d	DC 01	—	random1 23abc45 6def	—	—	198.5 1.100. 23	—	—
202 6- 02- 04 12: 02: 10	47 68	Kerb eros TGT requ este d	DC 01	—	random1 23abc45 6def	—	—	198.5 1.100. 23	—	—
202 6- 02- 04 12: 03: 30	47 69	Kerb eros servi ce ticke t requ este d	DC 01	—	random1 23abc45 6def	—	—	198.5 1.100. 23	—	—

Table 3.15 – Practical part

admin	created	suspicious_privileged_account	HIGH	75	2026-02-03T18:52:10+00:00	normal user	203.0.113.50	wsx01	identity, privilege, timing	Account creation/activation outside of business hours or on a weekend (+20) Account name similar to a service/privileged account: admin (+10) Account enabled shortly after creation/discovery (<=15 minutes) (+10) Addition to privileged groups: Domain Admins, Enterprise Admins (+35)
guest	created	suspicious_privileged_account	HIGH	75	2026-02-04T06:13:25+00:00	guest	10.2.3.4	ki0sk1	identity, privilege, timing	Account creation/activation outside of business hours or on a weekend (+20) Account name similar to a service/privileged account: guest (+10) Account enabled shortly after creation/discovery (<=15 minutes) (+10) Addition to privileged groups: Schema Admins (+35)
support	created	suspicious_privileged_account	HIGH	75	2026-02-03T20:06:15+00:00	help desk	10.0.1.12	ws3	identity, privilege, timing	Account creation/activation outside of business hours or on a weekend (+20) Account name appears to be a service/privileged account: support (+10) Account enabled shortly after creation/discovery (<=15 minutes) (+10)

testuser	created	suspicious_privileged_account	HIGH	70	2026-02-03T21:12:30+00:00	guest	10.2.3.4	kiosk1	identity, privilege, timing	Addition to privileged groups:
root	created	suspicious_privileged_account	MEDIUM	50	2026-02-04T10:02:10+00:00	normaluser			identity, privilege, timing	Creation/activation during business hours (-5) Account name similar to service/privileged account: root (+10) Account enabled shortly after creation/discovery (<=15 minutes) (+10) Addition to privileged groups: Enterprise Admins (+35)

random123abc456def (+10) | Account enabled shortly after creation/discovery (<=15 minutes) (+10) | Added to privileged groups: Administrators, Domain Admins (+35) | Kerberos TGT request detected shortly after account creation/activation (4768) (+5) | Accelerated chain of events detected after account creation/activation (+15)
 svc_backup — HIGH (90)
 Case origin: created
 Case type: suspicious_privileged_kerberos_account
 Created / activated: 2026-02-03T19:30:00+00:00
 Creator: itadmin

Creator source: IP=10.0.0.50; WS=jump01

Risk categories: identity, kerberos, privilege, timing

Reasons: The account name looks typical: svc_backup (-5) | Account creation/activation outside of business hours or on a weekend (+20) | Account enabled shortly after creation/discovery (<=15 minutes) (+10) | Added to privileged groups: Backup Operators (+35) | An attempt to change/reset a password was detected shortly after creation/activation (+10) | Kerberos TGT request was detected shortly after account creation/activation (4768) (+5) | An accelerated chain of events was detected after account creation/activation (+15)

admin — HIGH (75)

Case origin: created

Case type: suspicious_privileged_account

Created / activated: 2026-02-03T18:52:10+00:00 Creator: normaluser

Creator source: IP=203.0.113.50; WS=ws-ext01

Risk categories: identity, privilege, timing

Reasons: Account creation/activation outside business hours or on a weekend (+20) | The account name appears to be a service/privileged account: admin (+10)

Table 3.16 – Practical part

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-03 19:30:00	4720	User account created	DC01	itadmin	svc_backup	—	—	—	—	—
2026-02-03 19:33:50	4722	User account enabled	DC01	itadmin	svc_backup	—	—	—	—	—
2026-02-03 19:36:20	4756	Member added to security-enabled universal group	DC01	itadmin	—	Backup Operators	CN=svc_backup, DC=CONTOSO	—	—	—
2026-02-	4724	Attempt to reset	SERVER2	support	svc_backup	—	—	—	—	—

03 19:38:05		account password								
2026-02-03 19:40:10	4624	Successful login	DC01	—	svc_backup	—	—	8.8.8.8	WS1	3
2026-02-03 19:41:30	4768	Kerberos TGT requested	DC01	—	svc_backup	—	—	8.8.8.8	—	—

Account enabled shortly after creation/discovery (<=15 minutes) (+10) | Added to privileged groups: Domain Admins, Enterprise Admins (+35)

Table 3.17 – Practical part

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-03 18:52:10	4720	User account created	DC01	normaluser	admin	—	—	—	—	—
2026-02-03 18:58:20	4722	User account enabled	DC01	normaluser	admin	—	—	—	—	—
2026-02-03 19:00:10	4732	Member added to security-enabled local group	DC01	normaluser	—	Domain Admins	CN=admin,DC=CONTOSO	—	—	—
2026-02-03	4732	Member added to	DC01	normaluser	—	Enterprise Admin	CN=admin,DC=CONTOSO	—	—	—

19:01:30		security-enabled local group				ins				
2026-02-03 19:03:55	4625	Failed logon	DC01	—	admin	—	—	203.0.113.77	LAPTOP-HACK	3
2026-02-03 19:04:40	4625	Failed logon	DC01	—	admin	—	—	203.0.113.77	LAPTOP-HACK	3
2026-02-03 19:06:05	4624	Successful logon	DC01	—	admin	—	—	203.0.113.77	LAPTOP-HACK	10
2026-02-03 19:07:30	4768	Kerberos TGT requested	DC01	—	admin	—	—	203.0.113.77	—	—
2026-02-03 19:08:25	4769	Kerberos service ticket requested	DC01	—	admin	—	—	203.0.113.77	—	—
2026-02-04 02:00:00	4724	Attempt to reset account password	SERVER2	random123abc456def	admin	—	—	—	—	—
2026-02-04 02:02:10	4724	Attempt to reset account	SERVER2	root	admin	—	—	—	—	—

		pass word								
202 6- 02- 04 02:0 5:00	462 5	Faile d logon	SER VER 2	—	admi n	—	—	1.1.1. 1	PC- UNK NOW N	3
202 6- 02- 04 02:0 6:30	462 4	Succ essful logon	SER VER 2	—	admi n	—	—	1.1.1. 1	PC- UNK NOW N	10
202 6- 02- 04 12:0 0:00	476 8	Kerb eros TGT reque sted	DC0 1	—	admi n	—	—	203.0. 113.7 7	—	—
202 6- 02- 04 12:0 1:05	476 9	Kerb eros servi ce ticket reque sted	DC0 1	—	admi n	—	—	203.0. 113.7 7	—	—

guest — HIGH (75)

Case origin: created

Case type: suspicious_privileged_account

Created / activated: 2026-02-04T06:13:25+00:00

Creator: guest

Creator source: IP=10.2.3.4; WS=kiosk1

Risk categories: identity, privilege, timing

Reasons: Account creation/activation outside business hours or on a weekend (+20) | The account name appears to be a service/privileged one: guest (+10) |

The account is enabled shortly after creation/discovery (<=15 minutes) (+10) | Adding to privileged groups: Schema Admins (+35)

Table 3.18 – Practical part

Tim e	Eve ntID	Descri ption	Ho st	Subjec tUser	Targe tUser	Gro up	Member	IP	Works tation	Logon Type
2026 -02- 04 06:1 3:25	472 0	User accou nt create d	D C0 1	guest	guest	—	—	—	—	—
2026	472	User	D	guest	guest	—	—	—	—	—

-02-04 06:20:05	2	account enabled	C01							
2026-02-04 06:22:40	4728	Member added to security-enabled global group	DC01	admin	—	Schema Admins	CN=guest,DC=CONTOSO	—	—	—
2026-02-04 06:25:00	4624	Successful logon	DC01	—	guest	—	—	203.0.113.50	WS-EXT01	10

support — HIGH (75)

Case origin: created

Case type: suspicious_privileged_account

Created / activated: 2026-02-03T20:06:15+00:00

Creator: helpdesk

Creator source: IP=10.0.1.12; WS=ws3

Risk categories: identity, privilege, timing

Reasons: Account creation/activation outside business hours or on a weekend (+20) | The account name appears to be a service/privileged one: support (+10) |

The account is enabled shortly after creation/discovery (<=15 minutes) (+10) | Adding to privileged groups: Enterprise Admins (+35)

Table 3.19 – Practical part

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-03 20:06:15	4720	User account created	WORKSTATION1	helpdesk	support	—	—	—	—	—
2026-02-03 20:12:	4722	User account enabled	WORKSTATION1	helpdesk	support	—	—	—	—	—

10										
2026-02-03 20:14:30	4625	Failed logon	DC01	—	support	—	—	172.16.0.10	LAPTOP3	3
2026-02-03 20:16:00	4624	Successful logon	SERVER2	—	support	—	—	192.168.1.100	WS2	10
2026-02-03 20:18:10	4732	Member added to security-enabled local group	DC01	suspectuser	—	Enterprise Admins	CN=support,DC=FABRIKAM	—	—	—

testuser — HIGH (70)

Case origin: created

Case type: suspicious_privileged_account

Created / activated: 2026-02-03T21:12:30+00:00

Creator: guest

Creator source: IP=10.2.3.4; WS=kiosk1

Risk categories: identity, privilege, timing

Reasons: The account name looks generic: testuser (-5) |

Account creation/activation outside business hours or on a weekend (+20) | The account is enabled shortly after creation/discovery (<=15 minutes) (+10) | Adding to privileged groups:Schema Admins (+35) | An attempt to change/reset the password was recorded shortly after creation/activation (+10)

Table 3.20 – Practical part

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-03 21:1	4720	User account created	WORKSTATION1	guest	testuser	—	—	—	—	—

2:30										
202 6- 02- 03 21:1 8:05	472 2	User accou nt enabl ed	WORKST ATION1	guest	testus er	—	—	—	—	—
202 6- 02- 03 21:2 0:00	472 8	Mem ber added to securi ty- enabl ed global group	WORKST ATION1	testus er	—	Sch ema Ad min s	CN=testuser,D C=ADATUM	—	—	—
202 6- 02- 03 21:2 2:10	462 4	Succe ssful logon	DC01	—	testus er	—	—	8.8 .8. 8	LAPT OP7	10
202 6- 02- 03 21:2 4:00	472 4	Attem pt to reset accou nt passw ord	DC01	svc_b ackup	testus er	—	—	—	—	—

root — MEDIUM (50)

Case origin: created

Case type: suspicious_privileged_account

Created / activated: 2026-02-04T10:02:10+00:00

Creator: normaluser

Creator source: IP=; WS=

Risk categories: identity, privilege, timing

Reasons: Creation/activation during business hours (-5) | Account name similar to service/privileged: root (+10)|

Account enabled shortly after creation/discovery (<=15 minutes) (+10) | Added to privileged groups: Enterprise Admins (+35)

Table 3.21 – Practical part

Time	EventID	Description	Host	SubjectUser	TargetUser	Group	Member	IP	Workstation	LogonType
2026-02-04 10:02:10	4720	User account created	WORKSTATION1	normaluser	root	—	—	—	—	—
2026-02-04 10:06:50	4722	User account enabled	WORKSTATION1	normaluser	root	—	—	—	—	—
2026-02-04 10:08:20	4732	Member added to security-enabled local group	DC01	admin	—	Enterprise Admins	CN=root,DC=ADATUM	—	—	—
2026-02-04 10:10:30	4625	Failed logon	DC01	—	root	—	—	172.16.0.10	WS2	3
2026-02-04 10:12:40	4624	Successful logon	DC01	—	root	—	—	172.16.0.10	WS2	10

Also Findings.

```
"account": "random123abc456def",
  "created_at": "2026-02-03T19:12:10+00:00",
  "case_origin": "created",
  "case_type": "suspicious_privileged_kerberos_account",
  "risk_score": 95,
  "risk_level": "HIGH",
  "risk_categories": [
    "identity",
    "kerberos",
```

```

    "privilege",
    "timing"
  ],
  "creator": "suspectuser",
  "creator_ip": "192.168.1.100",
  "creator_workstation": "ws2",
  "reasons": [
    " Account creation/activation outside business hours or on a weekend (+20)",
    "The account name appears to be automatically generated:
random123abc456def (+10)",
    " Account enabled shortly after creation/discovery (<=15 minutes) (+10)",
    " Adding to privileged groups: Administrators, Domain Admins (+35)",
    " Kerberos TGT request detected shortly after account creation/activation
(4768) (+5)",
    " Accelerated chain of events detected after account creation/activation (+15)"
  ],
  "timeline": [
    {
      "ts": "2026-02-03T19:12:10+00:00",
      "event_id": 4720,
      "event_desc": "User account created",
      "host": "SERVER2",
      "subject_user": "suspectuser",
      "target_user": "random123abc456def",
      "group_name": null,
      "member_name": null,
      "ip_address": null,
      "workstation": null,
      "logon_type": null,
      "process_name": null,
      "object_class": null,
      "object_dn": null,
      "attribute_name": null,
      "operation_type": null
    },
  ],

```

CONCLUSION

The study found that improving the detection of fake accounts in an Active Directory-based corporate network requires the use of a combination of several threat detection methods. Taken individually, the systematic analysis of logs and the identification of behavioral anomalies are not sufficient tools to identify fake accounts. Therefore, only the comprehensive application of correlation methods for event processing, together with the analysis of protocols and the detection of anomalies, provides truly reliable detection.

The goal of the study has been achieved. To achieve the goal, we had to solve several tasks at once, and all of them were closely interrelated. They had to theoretically analyze the role of Active Directory in security, evaluate and study existing approaches to detecting threats, highlight diagnostic signs for detecting a fake account, and finally develop an algorithm for automated search. In the course of the work, the most important data sources were identified and, accordingly, an understandable method for processing this data was proposed.

As a result of the thesis, a working prototype of the system for detecting fake accounts was created. The system can collect logs, parse, normalize and analyze the event log in JSON/JSONL format. The conducted check confirms its ability to identify suspicious scenarios: the unauthorized creation of accounts, uncharacteristic for actions of the user, suspicious switching on old accounts. Python was selected as the implementation language. This choice is due to the flexibility of the language, which means that if necessary it can be easily changed without a complete adaptation of the architecture.

The practical significance of the work is that the developed prototype can be used for actual monitoring of domain events and early detection of potential threats in an enterprise environment, especially in organizations that do not have expensive SIEM solutions.

As recommendations for further improvement of the system, it is proposed to expand the number of analyzed events with additional event IDs, introduce machine learning methods to improve the accuracy of detection and reduce the number of false positives, as well as enable integration into existing monitoring and response platforms.

REFERENCES

- [1] Microsoft, “Active Directory Domain Services overview,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview>. [Accessed: May 7, 2026].
- [2] Microsoft, “Windows Authentication overview,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/security/windows-authentication/windows-authentication-overview>. [Accessed: May 7, 2026].
- [3] Microsoft, “Kerberos authentication overview in Windows Server,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/security/kerberos/kerberos-authentication-overview>. [Accessed: May 7, 2026].
- [4] Microsoft, “[MS-KILE]: Kerberos Protocol Extensions,” Microsoft Learn, 2025. [Online]. Available: https://learn.microsoft.com/enus/openspecs/windows_protocols/ms-kile/2a32282e-dd48-4ad9-a542-609804b02cc9. [Accessed: May 7, 2026].
- [5] C. Neuman, T. Yu, S. Hartman, and K. Raeburn, “The Kerberos Network Authentication Service (V5),” RFC 4120, Internet Engineering Task Force, 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4120>. [Accessed: May 7, 2026].
- [6] Microsoft, “Security identifiers,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/understand-security-identifiers>. [Accessed: May 7, 2026].
- [7] Microsoft, “Audit User Account Management,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-user-account-management>. [Accessed: May 7, 2026].
- [8] Microsoft, “Audit Security Group Management,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-security-group-management>. [Accessed: May 7, 2026].
- [9] Microsoft, “4624(S): An account was successfully logged on,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4624>. [Accessed: May 7, 2026].
- [10] Microsoft, “4625(F): An account failed to log on,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625>. [Accessed: May 7, 2026].
- [11] Microsoft, “4648(S): A logon was attempted using explicit credentials,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4648>. [Accessed: May 7, 2026].
- [12] Microsoft, “4672(S): Special privileges assigned to new logon,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4672>. [Accessed: May 7, 2026].
- [13] Microsoft, “4688(S): A new process has been created,” Microsoft Learn,

2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4688>. [Accessed: May 7, 2026].

[14] Microsoft, “4720(S): A user account was created,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4720>. [Accessed: May 7, 2026].

[15] Microsoft, “4722(S): A user account was enabled,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4722>. [Accessed: May 7, 2026].

[16] Microsoft, “4723(S, F): An attempt was made to change an account’s password,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4723>. [Accessed: May 7, 2026].

[17] Microsoft, “4724(S, F): An attempt was made to reset an account’s password,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4724>. [Accessed: May 7, 2026].

[18] Microsoft, “4725(S): A user account was disabled,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4725>. [Accessed: May 7, 2026].

[19] Microsoft, “4726(S): A user account was deleted,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4726>. [Accessed: May 7, 2026].

[20] Microsoft, “4728(S): A member was added to a security-enabled global group,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4728>. [Accessed: May 7, 2026].

[21] Microsoft, “4732(S): A member was added to a security-enabled local group,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4732>. [Accessed: May 7, 2026].

[22] Microsoft, “4738(S): A user account was changed,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4738>. [Accessed: May 7, 2026].

[23] Microsoft, “4740(S): A user account was locked out,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4740>. [Accessed: May 7, 2026].

[24] Microsoft, “4756(S): A member was added to a security-enabled universal group,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4756>. [Accessed: May 7, 2026].

[25] Microsoft, “4768(S, F): A Kerberos authentication ticket (TGT) was requested,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4768>. [Accessed: May 7, 2026].

[26] Microsoft, “4769(S, F): A Kerberos service ticket was requested,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4769>. [Accessed: May 7, 2026].

- [27] Microsoft, “4771(F): Kerberos pre-authentication failed,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4771>. [Accessed: May 7, 2026].
- [28] Microsoft, “4776(S, F): The computer attempted to validate the credentials for an account,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4776>. [Accessed: May 7, 2026].
- [29] Microsoft, “5136(S): A directory service object was modified,” Microsoft Learn, 2021. [Online]. Available: <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-5136>. [Accessed: May 7, 2026].
- [30] M. Russinovich and T. Garnier, “Sysmon,” Microsoft Sysinternals, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>. [Accessed: May 7, 2026].
- [31] Elastic, “Winlogbeat,” Elastic Documentation, 2026. [Online]. Available: <https://www.elastic.co/docs/reference/beats/winlogbeat>. [Accessed: May 7, 2026].
- [32] Microsoft, “What is Microsoft Sentinel?,” Microsoft Learn, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/sentinel/sentinel-overview>. [Accessed: May 7, 2026].
- [33] Splunk, “SIEM: Security Information and Event Management explained,” Splunk, 2026. [Online]. Available: <https://www.splunk.com/en-us/blog/learn/siem-security-information-event-management.html>. [Accessed: May 7, 2026].
- [34] IBM, “IBM QRadar SIEM 7.5.0 documentation,” IBM Documentation, 2026. [Online]. Available: https://www.ibm.com/docs/en/SS42VS_7.5/com.ibm.qradar.doc/c_qradar_pdfs.html. [Accessed: May 7, 2026].
- [35] Elastic, “About detection rules,” Elastic Security Documentation, 2026. [Online]. Available: <https://www.elastic.co/docs/solutions/security/detect-and-alert/about-detection-rules>. [Accessed: May 7, 2026].
- [36] SigmaHQ, “Sigma — Generic Signature Format for SIEM Systems,” SigmaHQ, 2026. [Online]. Available: <https://sigmahq.io/sigma/>. [Accessed: May 7, 2026].
- [37] SigmaHQ, “Sigma Specification,” SigmaHQ, 2026. [Online]. Available: <https://sigmahq.io/sigma-specification/>. [Accessed: May 7, 2026].
- [38] MITRE, “Create Account, Technique T1136,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1136/>. [Accessed: May 7, 2026].
- [39] MITRE, “Create Account: Domain Account, Sub-technique T1136.002,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1136/002/>. [Accessed: May 7, 2026].
- [40] MITRE, “Account Discovery: Domain Account, Sub-technique T1087.002,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1087/002/>. [Accessed: May 7, 2026].
- [41] MITRE, “Valid Accounts, Technique T1078,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1078/>. [Accessed: May 7, 2026].

[42] MITRE, “Steal or Forge Kerberos Tickets, Technique T1558,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1558/>. [Accessed: May 7, 2026].

[43] MITRE, “Steal or Forge Kerberos Tickets: Golden Ticket, Sub-technique T1558.001,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1558/001/>. [Accessed: May 7, 2026].

[44] MITRE, “Steal or Forge Kerberos Tickets: Kerberoasting, Sub-technique T1558.003,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/techniques/T1558/003/>. [Accessed: May 7, 2026].

[45] MITRE, “Detect Kerberoasting Attempts (T1558.003), Detection Strategy DET0157,” MITRE ATT&CK, 2025. [Online]. Available: <https://attack.mitre.org/detectionstrategies/DET0157/>. [Accessed: May 7, 2026].

[46] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” NIST Special Publication 800-92, National Institute of Standards and Technology, 2006. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-92>. [Accessed: May 7, 2026].

[47] A.Nelson, S.Rekhi, K.Scarfone, and M.Souppaya, “Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile,” NIST Special Publication 800-61 Revision 3, National Institute of Standards and Technology, 2025. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-61r3>. [Accessed: May 7, 2026].

[48] C. Pascoe, S. Quinn, and K. Scarfone, “The NIST Cybersecurity Framework (CSF) 2.0,” NIST Cybersecurity White Paper 29, National Institute of Standards and Technology, 2024. [Online].?Available: <https://doi.org/10.6028/NIST.CSWP.29>. [Accessed: May 7, 2026].

[49] Center for Internet Security, “CIS Critical Security Controls Version 8.1,” CIS, 2024. [Online]. Available: <https://www.cisecurity.org/controls/v8-1>. [Accessed: May 7, 2026].

[50] OWASP Foundation, “OWASP Top 10:2021,” OWASP, 2021. [Online]. Available: <https://owasp.org/Top10/2021/>. [Accessed: May 7, 2026].

APPENDIX

Code listing.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

AD Fake Account Detector (single-file, rules + scoring + correlation).

Purpose:

Defensive analysis of exported Windows Security logs / JSONL events to detect suspicious newly created or suspiciously enabled AD accounts.

Input:

JSON Lines (one JSON object per line) or JSON array with Windows Security events
(Winlogbeat / SIEM / Sentinel / custom export preferred).

Output:

Timestamped report folder with:

- summary.csv
- findings.json
- report.html

Notes:

- This tool is defensive. It analyzes logs only.
- It does NOT authenticate, escalate privileges, or perform active exploitation.
- To collect live Security logs outside exports you may need administrative rights, but this script itself works on exported data.

Recommended scenarios:

- new account created outside business hours
- rapid enablement and privileged group assignment
- first logon from non-corporate IP
- creator not in allow-list
- burst creation by same creator
- Kerberos activity shortly after account creation/enablement

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import csv
import html
```

```

import ipaddress
import json
import re
from collections import Counter, defaultdict
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Set, Tuple

```

```

#

```

```

=====
# Event model / config
#
=====

```

```

EVENT_DESCR = {
    4624: "Successful logon",
    4625: "Failed logon",
    4648: "Logon attempted using explicit credentials",
    4672: "Special privileges assigned to new logon",
    4688: "Process creation",
    4720: "User account created",
    4722: "User account enabled",
    4723: "Attempt to change account password",
    4724: "Attempt to reset account password",
    4725: "User account disabled",
    4726: "User account deleted",
    4728: "Member added to security-enabled global group",
    4732: "Member added to security-enabled local group",
    4738: "User account was changed",
    4740: "User account locked out",
    4756: "Member added to security-enabled universal group",
    4768: "Kerberos TGT requested",
    4769: "Kerberos service ticket requested",
    4771: "Kerberos pre-authentication failed",
    4776: "NTLM authentication attempted",
    5136: "Directory Service object modified",
}

```

```

SUPPORTED_EVENT_IDS = set(EVENT_DESCR.keys())

```

```

PRIV_GROUP_DEFAULT = {
    "domain admins",
    "enterprise admins",
}

```

```

    "administrators",
    "account operators",
    "backup operators",
    "schema admins",
    "server operators",
    "print operators",
    "dnsadmins",
}

# suspicious role-like names
SUSP_NAME_RE = re.compile(

r"(?i)^(?:admin|administrator|root|support|helpdesk|svc|service|backup|sql|oracle|test|t
emp|guest|user)\b"
)

# optional org-specific "normal-ish" name pattern can be expanded later
NORMAL_NAME_HINT_RE = re.compile(r"^[A-Za-z][A-Za-z0-9._-
]{2,32}$")

DEFAULT_WEIGHTS = {
    # positive risk
    "offhours": 20,
    "creator_not_allowed": 15,
    "creator_host_not_allowed": 10,
    "susp_name": 10,
    "rand_name": 10,
    "enabled_soon": 10,
    "priv_group": 35,
    "special_priv": 20,
    "first_logon_ext_ip": 15,
    "many_fails": 10,
    "explicit_creds": 10,
    "pwd_reset_soon": 10,
    "acct_changed_soon": 10,
    "acct_disabled_soon": 5,
    "acct_deleted_soon": 15,
    "lockout_after_create": 10,
    "kerberos_tgt_soon": 5,
    "kerberos_tgs_burst": 10,
    "kerberos_failures": 10,
    "ntlm_attempts": 5,
    "ds_object_changed": 10,
    "burst_creator": 20,

```

```

"activity_chain": 15,
"enabled_without_create": 15,

# negative / normalization
"creator_allowed_bonus": -10,
"creator_host_allowed_bonus": -5,
"business_hours_bonus": -5,
"corp_ip_bonus": -5,
"normal_name_bonus": -5,
}

```

```

GROUP_ADD_EVENT_IDS = {4728, 4732, 4756}
CASE_WINDOW_HOURS_DEFAULT = 24
BURST_WINDOW_MINUTES_DEFAULT = 10
BURST_COUNT_THRESHOLD_DEFAULT = 3

```

```

#

```

```

=====
# Helpers
#
=====

```

```

def _try_int(x: Any) -> Optional[int]:
    try:
        return int(x)
    except Exception:
        return None

```

```

def norm_str(x: Any) -> Optional[str]:
    if x is None:
        return None
    s = str(x).strip()
    return s if s else None

```